

Capítulo 4

Análisis sintáctico

Este capítulo está dedicado a los métodos de análisis sintáctico que se utilizan, por lo general, en los compiladores. Primero presentaremos los conceptos básicos, después las técnicas adecuadas para la implementación manual y, por último, los algoritmos que se han utilizado en las herramientas automatizadas. Debido a que los programas pueden contener errores sintácticos, hablaremos sobre las extensiones de los métodos de análisis sintáctico para recuperarse de los errores comunes.

Por diseño, cada lenguaje de programación tiene reglas precisas, las cuales prescriben la estructura sintáctica de los programas bien formados. Por ejemplo, en C un programa está compuesto de funciones, una función de declaraciones e instrucciones, una instrucción de expresiones, y así sucesivamente. La sintaxis de las construcciones de un lenguaje de programación puede especificarse mediante gramáticas libres de contexto o la notación BNF (Forma de Backus-Naur), que se presentó en la sección 2.2. Las gramáticas ofrecen beneficios considerables, tanto para los diseñadores de lenguajes como para los escritores de compiladores.

- Una gramática proporciona una especificación sintáctica precisa, pero fácil de entender, de un lenguaje de programación.
- A partir de ciertas clases de gramáticas, podemos construir de manera automática un analizador sintáctico eficiente que determine la estructura sintáctica de un programa fuente. Como beneficio colateral, el proceso de construcción del analizador sintáctico puede revelar ambigüedades sintácticas y puntos problemáticos que podrían haberse pasado por alto durante la fase inicial del diseño del lenguaje.
- La estructura impartida a un lenguaje mediante una gramática diseñada en forma apropiada es útil para traducir los programas fuente en código objeto correcto, y para detectar errores.
- Una gramática permite que un lenguaje evolucione o se desarrolle en forma iterativa, agregando nuevas construcciones para realizar nuevas tareas. Estas nuevas construcciones pueden integrarse con más facilidad en una implementación que siga la estructura gramatical del lenguaje.

4.1 Introducción

En esta sección, examinaremos la forma en que el analizador sintáctico se acomoda en un compilador ordinario. Después analizaremos las gramáticas comunes para las expresiones aritméticas. Las gramáticas para las expresiones son suficientes para ilustrar la esencia del análisis sintáctico, ya que dichas técnicas de análisis para las expresiones se transfieren a la mayoría de las construcciones de programación. Esta sección termina con una explicación sobre el manejo de errores, ya que el analizador sintáctico debe responder de manera adecuada para descubrir que su entrada no puede ser generada por su gramática.

4.1.1 La función del analizador sintáctico

En nuestro modelo de compilador, el analizador sintáctico obtiene una cadena de tokens del analizador léxico, como se muestra en la figura 4.1, y verifica que la cadena de nombres de los tokens pueda generarse mediante la gramática para el lenguaje fuente. Esperamos que el analizador sintáctico reporte cualquier error sintáctico en forma inteligible y que se recupere de los errores que ocurren con frecuencia para seguir procesando el resto del programa. De manera conceptual, para los programas bien formados, el analizador sintáctico construye un árbol de análisis sintáctico y lo pasa al resto del compilador para que lo siga procesando. De hecho, el árbol de análisis sintáctico no necesita construirse en forma explícita, ya que las acciones de comprobación y traducción pueden intercalarse con el análisis sintáctico, como veremos más adelante. Por ende, el analizador sintáctico y el resto de la interfaz de usuario podrían implementarse sin problemas mediante un solo módulo.

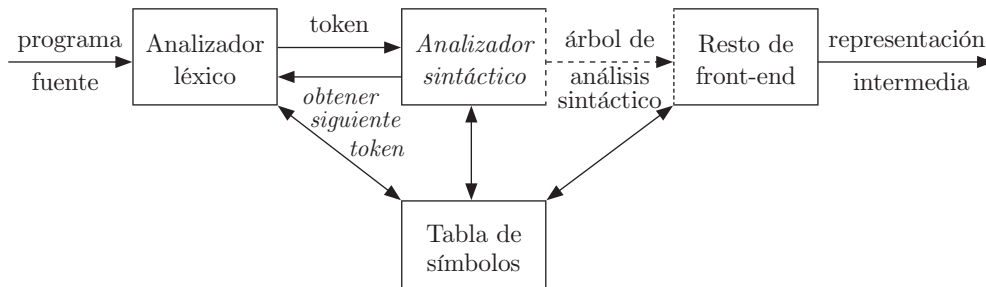


Figura 4.1: Posición del analizador sintáctico en el modelo del compilador

Existen tres tipos generales de analizadores para las gramáticas: universales, descendentes y ascendentes. Los métodos universales de análisis sintáctico como el algoritmo de Cocke-Younger-Kasami y el algoritmo de Earley pueden analizar cualquier gramática (vea las notas bibliográficas). Sin embargo, estos métodos generales son demasiado ineficientes como para usarse en la producción de compiladores.

Los métodos que se utilizan, por lo regular, en los compiladores pueden clasificarse como descendentes o ascendentes. Según sus nombres, los métodos descendentes construyen árboles de análisis sintáctico de la parte superior (raíz) a la parte inferior (hojas), mientras que los métodos ascendentes empiezan de las hojas y avanzan hasta la raíz. En cualquier caso, la entrada al analizador se explora de izquierda a derecha, un símbolo a la vez.

Los métodos descendentes y ascendentes más eficientes sólo funcionan para subclases de gramáticas, pero varias de estas clases, en especial las gramáticas LL y LR, son lo bastante expresivas como para describir la mayoría de las construcciones sintácticas en los lenguajes de programación modernos. Los analizadores sintácticos que se implementan en forma manual utilizan con frecuencia gramáticas LL; por ejemplo, el método de análisis sintáctico predictivo de la sección 2.4.2 funciona para las gramáticas LL. Los analizadores para la clase más extensa de gramáticas LR, por lo general, se construyen mediante herramientas automatizadas.

En este capítulo vamos a suponer que la salida del analizador sintáctico es cierta representación del árbol de análisis sintáctico para el flujo de tokens que proviene del analizador léxico. En la práctica, hay una variedad de tareas que podrían realizarse durante el análisis sintáctico, como la recolección de información de varios tokens para colocarla en la tabla de símbolos, la realización de la comprobación de tipos y otros tipos de análisis semántico, así como la generación de código intermedio. Hemos agrupado todas estas actividades en el cuadro con el título “Resto del front-end” de la figura 4.1. En los siguientes capítulos cubriremos con detalle estas actividades.

4.1.2 Representación de gramáticas

Algunas de las gramáticas que examinaremos en este capítulo se presentan para facilitar la representación de gramáticas. Las construcciones que empiezan con palabras clave como **while** o **int** son muy fáciles de analizar, ya que la palabra clave guía la elección de la producción gramatical que debe aplicarse para hacer que coincida con la entrada. Por lo tanto, nos concentraremos en las expresiones, que representan un reto debido a la asociatividad y la precedencia de operadores.

La asociatividad y la precedencia se resuelvan en la siguiente gramática, que es similar a las que utilizamos en el capítulo 2 para describir expresiones, términos y factores. E representa a las expresiones que consisten en términos separados por los signos $+$, T representa a los términos que consisten en factores separados por los signos $*$, y F representa a los factores que pueden ser expresiones entre paréntesis o identificadores:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \tag{4.1}$$

La gramática para expresiones (4.1) pertenece a la clase de gramáticas LR que son adecuadas para el análisis sintáctico ascendentes. Esta gramática puede adaptarse para manejar operadores adicionales y niveles adicionales de precedencia. Sin embargo, no puede usarse para el análisis sintáctico descendente, ya que es recursiva por la izquierda.

La siguiente variante no recursiva por la izquierda de la gramática de expresiones (4.1) se utilizará para el análisis sintáctico descendente:

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \epsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned} \tag{4.2}$$

La siguiente gramática trata a los signos $+$ y $*$ de igual forma, de manera que sirve para ilustrar las técnicas para el manejo de ambigüedades durante el análisis sintáctico:

$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id} \quad (4.3)$$

Aquí, E representa a las expresiones de todo tipo. La gramática (4.3) permite más de un árbol de análisis sintáctico para las expresiones como $a + b * c$.

4.1.3 Manejo de los errores sintácticos

El resto de esta sección considera la naturaleza de los errores sintácticos y las estrategias generales para recuperarse de ellos. Dos de estas estrategias, conocidas como recuperaciones en modo de pánico y a nivel de frase, se describirán con más detalle junto con los métodos específicos de análisis sintáctico.

Si un compilador tuviera que procesar sólo programas correctos, su diseño e implementación se simplificaría en forma considerable. No obstante, se espera que un compilador ayude al programador a localizar y rastrear los errores que, de manera inevitable, se infiltran en los programas, a pesar de los mejores esfuerzos del programador. Aunque parezca increíble, son pocos los lenguajes que se diseñan teniendo en mente el manejo de errores, aun cuando éstos son tan comunes. Nuestra civilización sería radicalmente distinta si los lenguajes hablados tuvieran los mismos requerimientos en cuanto a precisión sintáctica que los lenguajes de computadora. La mayoría de las especificaciones de los lenguajes de programación no describen la forma en que un compilador debe responder a los errores; el manejo de los mismos es responsabilidad del diseñador del compilador. La planeación del manejo de los errores desde el principio puede simplificar la estructura de un compilador y mejorar su capacidad para manejar los errores.

Los errores de programación comunes pueden ocurrir en muchos niveles distintos.

- Los errores *léxicos* incluyen la escritura incorrecta de los identificadores, las palabras clave o los operadores; por ejemplo, el uso de un identificador `tamanoElipce` en vez de `tamanoElipse`, y la omisión de comillas alrededor del texto que se debe interpretar como una cadena.
- Los errores *sintácticos* incluyen la colocación incorrecta de los signos de punto y coma, además de llaves adicionales o faltantes; es decir, “{” o “}”. Como otro ejemplo, en C o Java, la aparición de una instrucción `case` sin una instrucción `switch` que la encierre es un error sintáctico (sin embargo, por lo general, esta situación la acepta el analizador sintáctico y se atrapa más adelante en el procesamiento, cuando el compilador intenta generar código).
- Los errores *semánticos* incluyen los conflictos de tipos entre los operadores y los operandos. Un ejemplo es una instrucción `return` en un método de Java, con el tipo de resultado `void`.
- Los errores *lógicos* pueden ser cualquier cosa, desde un razonamiento incorrecto del programador en el uso (en un programa en C) del operador de asignación `=`, en vez del operador de comparación `==`. El programa que contenga `=` puede estar bien formado; sin embargo, tal vez no refleje la intención del programador.

La precisión de los métodos de análisis sintáctico permite detectar los errores sintácticos con mucha eficiencia. Varios métodos de análisis sintáctico, como los métodos LL y LR, detectan

un error lo más pronto posible; es decir, cuando el flujo de tokens que proviene del analizador léxico no puede seguirse analizando de acuerdo con la gramática para el lenguaje. Dicho en forma más precisa, tienen la *propiedad de prefijo viable*, lo cual significa que detectan la ocurrencia de un error tan pronto como ven un prefijo de la entrada que no puede completarse para formar una cadena válida en el lenguaje.

Otra de las razones para enfatizar la recuperación de los errores durante el análisis sintáctico es que muchos errores parecen ser sintácticos, sea cual fuere su causa, y se exponen cuando el análisis sintáctico no puede continuar. Algunos errores semánticos, como los conflictos entre los tipos, también pueden detectarse con eficiencia; sin embargo, la detección precisa de los errores semánticos y lógicos en tiempo de compilación es, por lo general, una tarea difícil.

El mango de errores en un analizador sintáctico tiene objetivos que son simples de declarar, pero difíciles de llevar a cabo:

- Reportar la presencia de errores con claridad y precisión.
- Recuperarse de cada error lo bastante rápido como para poder detectar los errores siguientes.
- Agregar una sobrecarga mínima al procesamiento de los programas correctos.

Por fortuna, los errores comunes son simples, y a menudo basta con un mecanismo simple para su manejo.

¿De qué manera un mango de errores debe reportar la presencia de un error? Como mínimo, debe reportar el lugar en el programa fuente en donde se detectó un error, ya que hay una buena probabilidad de que éste en sí haya ocurrido en uno de los pocos tokens anteriores. Una estrategia común es imprimir la línea del problema con un apuntador a la posición en la que se detectó el error.

4.1.4 Estrategias para recuperarse de los errores

Una vez que se detecta un error, ¿cómo debe recuperarse el analizador sintáctico? Aunque no hay una estrategia que haya demostrado ser aceptable en forma universal, algunos métodos pueden aplicarse en muchas situaciones. El método más simple es que el analizador sintáctico termine con un mensaje de error informativo cuando detecte el primer error. A menudo se descubren errores adicionales si el analizador sintáctico puede restaurarse a sí mismo, a un estado en el que pueda continuar el procesamiento de la entrada, con esperanzas razonables de que un mayor procesamiento proporcione información útil para el diagnóstico. Si los errores se apilan, es mejor para el compilador desistir después de exceder cierto límite de errores, que producir una molesta avalancha de errores “falsos”.

El resto de esta sección se dedica a las siguientes estrategias de recuperación de los errores: modo de pánico, nivel de frase, producciones de errores y corrección global.

Recuperación en modo de pánico

Con este método, al describir un error el analizador sintáctico descarta los símbolos de entrada, uno a la vez, hasta encontrar un conjunto designado de *tokens de sincronización*. Por lo general, los tokens de sincronización son delimitadores como el punto y coma o `}`, cuya función en

el programa fuente es clara y sin ambigüedades. El diseñador del compilador debe seleccionar los tokens de sincronización apropiados para el lenguaje fuente. Aunque la corrección en modo de pánico a menudo omite una cantidad considerable de entrada sin verificar errores adicionales, tiene la ventaja de ser simple y, a diferencia de ciertos métodos que consideraremos más adelante, se garantiza que no entrará en un ciclo infinito.

Recuperación a nivel de frase

Al descubrir un error, un analizador sintáctico puede realizar una corrección local sobre la entrada restante; es decir, puede sustituir un prefijo de la entrada restante por alguna cadena que le permita continuar. Una corrección local común es sustituir una coma por un punto y coma, eliminar un punto y coma extraño o insertar un punto y coma faltante. La elección de la corrección local se deja al diseñador del compilador. Desde luego que debemos tener cuidado de elegir sustituciones que no nos lleven hacia ciclos infinitos, como sería, por ejemplo, si siempre insertáramos algo en la entrada adelante del símbolo de entrada actual.

La sustitución a nivel de frase se ha utilizado en varios compiladores que reparan los errores, ya que puede corregir cualquier cadena de entrada. Su desventaja principal es la dificultad que tiene para arreglárselas con situaciones en las que el error actual ocurre antes del punto de detección.

Producciones de errores

Al anticipar los errores comunes que podríamos encontrar, podemos aumentar la gramática para el lenguaje, con producciones que generen las construcciones erróneas. Un analizador sintáctico construido a partir de una gramática aumentada por estas producciones de errores detecta los errores anticipados cuando se utiliza una producción de error durante el análisis sintáctico. Así, el analizador sintáctico puede generar diagnósticos de error apropiados sobre la construcción errónea que se haya reconocido en la entrada.

Corrección global

Lo ideal sería que un compilador hiciera la menor cantidad de cambios en el procesamiento de una cadena de entrada incorrecta. Hay algoritmos para elegir una secuencia mínima de cambios, para obtener una corrección con el menor costo a nivel global. Dada una cadena de entrada incorrecta x y una gramática G , estos algoritmos buscarán un árbol de análisis sintáctico para una cadena y relacionada, de tal forma que el número de inserciones, eliminaciones y modificaciones de los tokens requeridos para transformar a x en y sea lo más pequeño posible. Por desgracia, estos métodos son en general demasiado costosos para implementarlos en términos de tiempo y espacio, por lo cual estas técnicas sólo son de interés teórico en estos momentos.

Hay que observar que un programa casi correcto tal vez no sea lo que el programador tenía en mente. Sin embargo, la noción de la corrección con el menor costo proporciona una norma para evaluar las técnicas de recuperación de los errores, la cual se ha utilizado para buscar cadenas de sustitución óptimas para la recuperación a nivel de frase.

4.2 Gramáticas libres de contexto

En la sección 2.2 se presentaron las gramáticas para describir en forma sistemática la sintaxis de las construcciones de un lenguaje de programación, como las expresiones y las instrucciones. Si utilizamos una variable sintáctica *instr* para denotar las instrucciones, y una variable *expr* para denotar las expresiones, la siguiente producción:

$$instr \rightarrow \text{if} (expr) instr \text{ else } instr \quad (4.4)$$

especifica la estructura de esta forma de instrucción condicional. Entonces, otras producciones definen con precisión lo que es una *expr* y qué más puede ser una *instr*.

En esta sección repasaremos la definición de una gramática libre de contexto y presentaremos la terminología para hablar acerca del análisis sintáctico. En especial, la noción de derivaciones es muy útil para hablar sobre el orden en el que se aplican las producciones durante el análisis sintáctico.

4.2.1 La definición formal de una gramática libre de contexto

En la sección 2.2 vimos que una gramática libre de contexto (o simplemente gramática) consiste en terminales, no terminales, un símbolo inicial y producciones.

1. Los *terminales* son los símbolos básicos a partir de los cuales se forman las cadenas. El término “nombre de token” es un sinónimo de “terminal”; con frecuencia usaremos la palabra “token” en vez de terminal, cuando esté claro que estamos hablando sólo sobre el nombre del token. Asumimos que las terminales son los primeros componentes de los tokens que produce el analizador léxico. En (4.4), los terminales son las palabras reservadas **if** y **else**, y los símbolos “(” y “)”.
2. Los *no terminales* son variables sintácticas que denotan conjuntos de cadenas. En (4.4), *instr* y *expr* son no terminales. Los conjuntos de cadenas denotados por los no terminales ayudan a definir el lenguaje generado por la gramática. Los no terminales imponen una estructura jerárquica sobre el lenguaje, que representa la clave para el análisis sintáctico y la traducción.
3. En una gramática, un no terminal se distingue como el *símbolo inicial*, y el conjunto de cadenas que denota es el lenguaje generado por la gramática. Por convención, las producciones para el símbolo inicial se listan primero.
4. Las producciones de una gramática especifican la forma en que pueden combinarse los terminales y los no terminales para formar cadenas. Cada *producción* consiste en:
 - (a) Un no terminal, conocido como *encabezado* o *lado izquierdo* de la producción; esta producción define algunas de las cadenas denotadas por el encabezado.
 - (b) El símbolo $\rightarrow$. Algunas veces se ha utilizado $::=$ en vez de la flecha.
 - (c) Un *cuerpo* o *lado derecho*, que consiste en cero o más terminales y no terminales. Los componentes del cuerpo describen una forma en que pueden construirse las cadenas del no terminal en el encabezado.

Ejemplo 4.5: La gramática en la figura 4.2 define expresiones aritméticas simples. En esta gramática, los símbolos de los terminales son:

id + - * / ()

Los símbolos de los no terminales son *expresión*, *term* y *factor*, y *expresión* es el símbolo inicial. $\square$

$$\begin{array}{ll} \textit{expresión} & \rightarrow \textit{expresión} + \textit{term} \\ \textit{expresión} & \rightarrow \textit{expresión} - \textit{term} \\ \textit{expresión} & \rightarrow \textit{term} \\ \textit{term} & \rightarrow \textit{term} * \textit{factor} \\ \textit{term} & \rightarrow \textit{term} / \textit{factor} \\ \textit{term} & \rightarrow \textit{factor} \\ \textit{factor} & \rightarrow (\textit{expresión}) \\ \textit{factor} & \rightarrow \textbf{id} \end{array}$$

Figura 4.2: Gramática para las expresiones aritméticas simples

4.2.2 Convenciones de notación

Para evitar siempre tener que decir que “éstos son los terminales”, “éstos son los no terminales”, etcétera, utilizaremos las siguientes convenciones de notación para las gramáticas durante el resto de este libro:

1. Estos símbolos son terminales:
 - (a) Las primeras letras minúsculas del alfabeto, como *a*, *b*, *c*.
 - (b) Los símbolos de operadores como +, *, etcétera.
 - (c) Los símbolos de puntuación como paréntesis, coma, etcétera.
 - (d) Los dígitos 0, 1, ..., 9.
 - (e) Las cadenas en negrita como **id** o **if**, cada una de las cuales representa un solo símbolo terminal.
2. Estos símbolos son no terminales:
 - (a) Las primeras letras mayúsculas del alfabeto, como *A*, *B*, *C*.
 - (b) La letra *S* que, al aparecer es, por lo general, el símbolo inicial.
 - (c) Los nombres en cursiva y minúsculas, como *expr* o *instr*.
 - (d) Al hablar sobre las construcciones de programación, las letras mayúsculas pueden utilizarse para representar no terminales. Por ejemplo, los no terminales para las expresiones, los términos y los factores se representan a menudo mediante *E*, *T* y *F*, respectivamente.

3. Las últimas letras mayúsculas del alfabeto, como X , Y , Z , representan *símbolos gramaticales*; es decir, pueden ser no terminales o terminales.
4. Las últimas letras minúsculas del alfabeto, como u , v , ..., z , representan cadenas de terminales (posiblemente vacías).
5. Las letras griegas minúsculas α , β , γ , por ejemplo, representan cadenas (posiblemente vacías) de símbolos gramaticales. Por ende, una producción genérica puede escribirse como $A \rightarrow \alpha$, en donde A es el encabezado y α el cuerpo.
6. Un conjunto de producciones $A \rightarrow \alpha_1$, $A \rightarrow \alpha_2$, ..., $A \rightarrow \alpha_k$ con un encabezado común A (las llamaremos *producciones* A), puede escribirse como $A \rightarrow \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_k$. A α_1 , α_2 , ..., α_k les llamamos las *alternativas* para A .
7. A menos que se indique lo contrario, el encabezado de la primera producción es el símbolo inicial.

Ejemplo 4.6: Mediante estas convenciones, la gramática del ejemplo 4.5 puede describirse en forma concisa como:

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Las convenciones de notación nos indican que E , T y F son no terminales, y E es el símbolo inicial. El resto de los símbolos son terminales. $\square$

4.2.3 Derivaciones

La construcción de un árbol de análisis sintáctico puede hacerse precisa si tomamos una vista derivacional, en la cual las producciones se tratan como reglas de rescritura. Empezando con el símbolo inicial, cada paso de rescritura sustituye a un no terminal por el cuerpo de una de sus producciones. Esta vista derivacional corresponde a la construcción descendente de un árbol de análisis sintáctico, pero la precisión que ofrecen las derivaciones será muy útil cuando hablemos del análisis sintáctico ascendente. Como veremos, el análisis sintáctico ascendente se relaciona con una clase de derivaciones conocidas como derivaciones de “más a la derecha”, en donde el no terminal por la derecha se rescribe en cada paso.

Por ejemplo, considere la siguiente gramática, con un solo no terminal E , la cual agrega una producción $E \rightarrow - E$ a la gramática (4.3):

$$E \rightarrow E + E \mid E * E \mid - E \mid (E) \mid \text{id} \quad (4.7)$$

La producción $E \rightarrow - E$ significa que si E denota una expresión, entonces $- E$ debe también denotar una expresión. La sustitución de una sola E por $- E$ se describirá escribiendo lo siguiente:

$$E \Rightarrow -E$$

lo cual se lee como “ E deriva a $-E$ ”. La producción $E \rightarrow (E)$ puede aplicarse para sustituir cualquier instancia de E en cualquier cadena de símbolos gramaticales por (E) ; por ejemplo, $E * E \Rightarrow (E) * E$ o $E * E \Rightarrow E * (E)$. Podemos tomar una sola E y aplicar producciones en forma repetida y en cualquier orden para obtener una secuencia de sustituciones. Por ejemplo,

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\mathbf{id})$$

A dicha secuencia de sustituciones la llamamos una *derivación* de $-(\mathbf{id})$ a partir de E . Esta derivación proporciona la prueba de que la cadena $-(\mathbf{id})$ es una instancia específica de una expresión.

Para una definición general de la derivación, considere un no terminal A en la mitad de una secuencia de símbolos gramaticales, como en $\alpha A \beta$, en donde α y β son cadenas arbitrarias de símbolos gramaticales. Suponga que $A \rightarrow \gamma$ es una producción. Entonces, escribimos $\alpha A \beta \Rightarrow \alpha \gamma \beta$. El símbolo $\Rightarrow$ significa, “se deriva en un paso”. Cuando una secuencia de pasos de derivación $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ se rescribe como α_1 a α_n , decimos que α_1 *deriva* a α_n . Con frecuencia es conveniente poder decir, “deriva en cero o más pasos”. Para este fin, podemos usar el símbolo $\xRightarrow{*}$. Así,

1. $\alpha \xRightarrow{*} \alpha$, para cualquier cadena α .
2. Si $\alpha \xRightarrow{*} \beta$ y $\beta \Rightarrow \gamma$, entonces $\alpha \xRightarrow{*} \gamma$.

De igual forma, $\xRightarrow{+}$ significa “deriva en uno o más pasos”.

Si $S \xRightarrow{*} \alpha$, en donde S es el símbolo inicial de una gramática G , decimos que α es una *forma de frase* de G . Observe que una forma de frase puede contener tanto terminales como no terminales, y puede estar vacía. Un *enunciado* de G es una forma de frase sin símbolos no terminales. El *lenguaje generado* por una gramática es su conjunto de oraciones. Por ende, una cadena de terminales w está en $L(G)$, el lenguaje generado por G , si y sólo si w es un enunciado de G (o $S \xRightarrow{*} w$). Un lenguaje que puede generarse mediante una gramática se considera un *lenguaje libre de contexto*. Si dos gramáticas generan el mismo lenguaje, se consideran como *equivalentes*.

La cadena $-(\mathbf{id} + \mathbf{id})$ es un enunciado de la gramática (4.7), ya que hay una derivación

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\mathbf{id} + E) \Rightarrow -(\mathbf{id} + \mathbf{id}) \quad (4.8)$$

Las cadenas E , $-E$, $-(E)$, $\dots$, $-(\mathbf{id} + \mathbf{id})$ son todas formas de frases de esta gramática. Escribimos $E \xRightarrow{*} -(\mathbf{id} + \mathbf{id})$ para indicar que $-(\mathbf{id} + \mathbf{id})$ puede derivarse de E .

En cada paso de una derivación, hay dos elecciones por hacer. Debemos elegir qué no terminal debemos sustituir, y habiendo realizado esta elección, debemos elegir una producción con ese no terminal como encabezado. Por ejemplo, la siguiente derivación alternativa de $-(\mathbf{id} + \mathbf{id})$ difiere de la derivación (4.8) en los últimos dos pasos:

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \mathbf{id}) \Rightarrow -(\mathbf{id} + \mathbf{id}) \quad (4.9)$$

Cada no terminal se sustituye por el mismo cuerpo en las dos derivaciones, pero el orden de las sustituciones es distinto.

Para comprender la forma en que trabajan los analizadores sintácticos, debemos considerar las derivaciones en las que el no terminal que se va a sustituir en cada paso se elige de la siguiente manera:

1. En las derivaciones *por la izquierda*, siempre se elige el no terminal por la izquierda en cada de frase. Si $\alpha \Rightarrow \beta$ es un paso en el que se sustituye el no terminal por la izquierda en α , escribimos $\alpha \xRightarrow{lm} \beta$.
2. En las derivaciones *por la derecha*, siempre se elige el no terminal por la derecha; en este caso escribimos $\alpha \xRightarrow{rm} \beta$.

La derivación (4.8) es por la izquierda, por lo que puede describirse de la siguiente manera:

$$E \xRightarrow{lm} -E \xRightarrow{lm} -(E) \xRightarrow{lm} -(E + E) \xRightarrow{lm} -(\mathbf{id} + E) \xRightarrow{lm} -(\mathbf{id} + \mathbf{id})$$

Observe que (4.9) es una derivación por la derecha.

Si utilizamos nuestras convenciones de notación, cada paso por la izquierda puede escribirse como $wA\gamma \xRightarrow{lm} w\delta\gamma$, en donde w consiste sólo de terminales, $A \rightarrow \delta$ es la producción que se aplica, y γ es una cadena de símbolos gramaticales. Para enfatizar que α deriva a β mediante una derivación por la izquierda, escribimos $\alpha \xRightarrow{*lm} \beta$. Si $S \xRightarrow{*lm} \alpha$, decimos que α es una *forma de frase izquierda* de la gramática en cuestión.

Las análogas definiciones son válidas para las derivaciones por la derecha. A estas derivaciones se les conoce algunas veces como derivaciones *canónicas*.

4.2.4 Árboles de análisis sintáctico y derivaciones

Un árbol de análisis sintáctico es una representación gráfica de una derivación que filtra el orden en el que se aplican las producciones para sustituir los no terminales. Cada nodo interior de un árbol de análisis sintáctico representa la aplicación de una producción. El nodo interior se etiqueta con el no terminal A en el encabezado de la producción; los hijos del nodo se etiquetan, de izquierda a derecha, mediante los símbolos en el cuerpo de la producción por la que se sustituyó esta A durante la derivación.

Por ejemplo, el árbol de análisis sintáctico para $-(\mathbf{id} + \mathbf{id})$ en la figura 4.3 resulta de la derivación (4.8), así como de la derivación (4.9).

Las hojas de un árbol de análisis sintáctico se etiquetan mediante no terminales o terminales y, leídas de izquierda a derecha, constituyen una forma de frase, a la cual se le llama *producto* o *frontera* del árbol.

Para ver la relación entre las derivaciones y los árboles de análisis sintáctico, considere cualquier derivación $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \cdots \Rightarrow \alpha_n$, en donde α_1 es un sólo no terminal A . Para cada forma de frase α_i en la derivación, podemos construir un árbol de análisis sintáctico cuyo producto sea α_i . El proceso es una inducción sobre i .

BASE: El árbol para $\alpha_1 = A$ es un solo nodo, etiquetado como A .

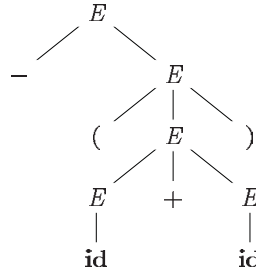


Figura 4.3: Árbol de análisis sintáctico para $-(\text{id} + \text{id})$

INDUCCIÓN: Suponga que ya hemos construido un árbol de análisis sintáctico con el producto $\alpha_{i-1} = X_1 X_2 \cdots X_k$ (tenga en cuenta que, de acuerdo a nuestras convenciones de notación, cada símbolo gramatical X_i es un no terminal o un terminal). Suponga que α_i se deriva de α_{i-1} al sustituir X_j , un no terminal, por $\beta = Y_1 Y_2 \cdots Y_m$. Es decir, en el i -ésimo paso de la derivación, la producción $X_j \rightarrow \beta$ se aplica a α_{i-1} para derivar $\alpha_i = X_1 X_2 \cdots X_{j-1} \beta X_{j+1} \cdots X_k$.

Para modelar este paso de la derivación, buscamos la j -ésima hoja, partiendo de la izquierda en el árbol de análisis sintáctico actual. Esta hoja se etiqueta como X_j . A esta hoja le damos m hijos, etiquetados $Y_1, Y_2, \dots, Y_m$, partiendo de la izquierda. Como caso especial, si $m = 0$ entonces $\beta = \epsilon$, y proporcionamos a la j -ésima hoja un hijo etiquetado como ϵ .

Ejemplo 4.10: La secuencia de árboles de análisis sintáctico que se construyen a partir de la derivación (4.8) se muestra en la figura 4.4. En el primer paso de la derivación, $E \Rightarrow -E$. Para modelar este paso, se agregan dos hijos, etiquetados como $-$ y E , a la raíz E del árbol inicial. El resultado es el segundo árbol.

En el segundo paso de la derivación, $-E \Rightarrow -(E)$. Por consiguiente, agregamos tres hijos, etiquetados como $(, E$ y $)$, al nodo hoja etiquetado como E del segundo árbol, para obtener el tercer árbol con coséchale producto $-(E)$. Si continuamos de esta forma, obtenemos el árbol de análisis sintáctico completo como el sexto árbol. $\square$

Como un árbol de análisis sintáctico ignora las variaciones en el orden en el que se sustituyen los símbolos en las formas de las oraciones, hay una relación de varios a uno entre las derivaciones y los árboles de análisis sintáctico. Por ejemplo, ambas derivaciones (4.8) y (4.9) se asocian con el mismo árbol de análisis sintáctico final de la figura 4.4.

En lo que sigue, realizaremos con frecuencia el análisis sintáctico produciendo una derivación por la izquierda o por la derecha, ya que hay una relación de uno a uno entre los árboles de análisis sintáctico y este tipo de derivaciones. Tanto las derivaciones por la izquierda como las de por la derecha eligen un orden específico para sustituir símbolos en las formas de las oraciones, por lo que también filtran las variaciones en orden. No es difícil mostrar que todos los árboles sintácticos tienen asociadas una derivación única por la izquierda y una derivación única por la derecha.

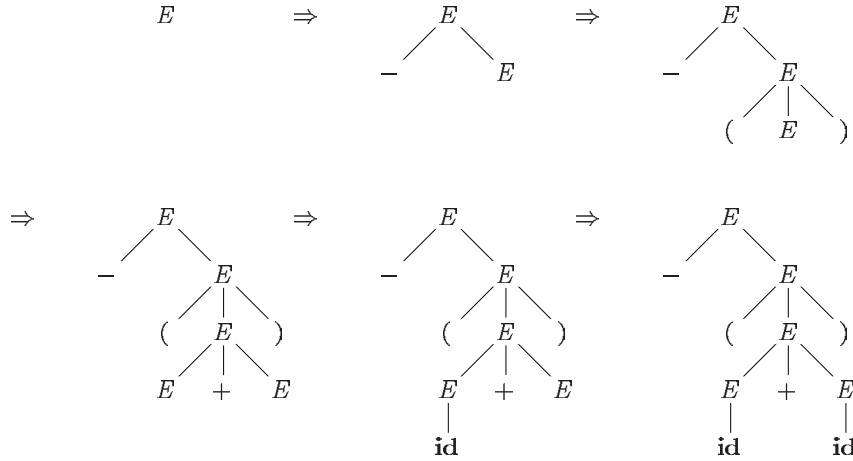


Figura 4.4: Secuencia de árboles de análisis sintáctico para la derivación (4.8)

4.2.5 Ambigüedad

En la sección 2.2.4 vimos que una gramática que produce más de un árbol de análisis sintáctico para cierto enunciado es *ambiguo*. Dicho de otra forma, una gramática ambigua es aquella que produce más de una derivación por la izquierda, o más de una derivación por la derecha para el mismo enunciado.

Ejemplo 4.11: La gramática de expresiones aritméticas (4.3) permite dos derivaciones por la izquierda distintas para el enunciado $\text{id} + \text{id} * \text{id}$:

$$\begin{array}{ll}
 E \Rightarrow E + E & E \Rightarrow E * E \\
 \Rightarrow \text{id} + E & \Rightarrow E + E * E \\
 \Rightarrow \text{id} + E * E & \Rightarrow \text{id} + E * E \\
 \Rightarrow \text{id} + \text{id} * E & \Rightarrow \text{id} + \text{id} * E \\
 \Rightarrow \text{id} + \text{id} * \text{id} & \Rightarrow \text{id} + \text{id} * \text{id}
 \end{array}$$

Los árboles de análisis sintáctico correspondientes aparecen en la figura 4.5.

Observe que el árbol de análisis sintáctico de la figura 4.5(a) refleja la precedencia que se asume comúnmente para $+$ y $*$, mientras que el árbol de la figura 4.5(b) no. Es decir, lo común es tratar al operador $*$ teniendo mayor precedencia que $+$, en forma correspondiente al hecho de que, por lo general, evaluamos la expresión $a + b * c$ como $a + (b * c)$, en vez de hacerlo como $(a + b) * c$. $\square$

Para la mayoría de los analizadores sintácticos, es conveniente que la gramática no tenga ambigüedades, ya que de lo contrario, no podemos determinar en forma única qué árbol de análisis sintáctico seleccionar para un enunciado. En otros casos, es conveniente usar gramáticas ambiguas elegidas con cuidado, junto con *reglas para eliminar la ambigüedad*, las cuales “descartan” los árboles sintácticos no deseados, dejando sólo un árbol para cada enunciado.

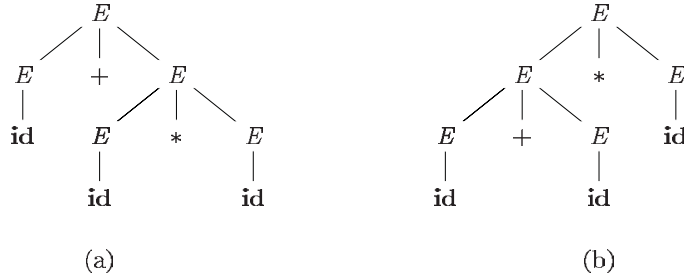


Figura 4.5: Dos árboles de análisis sintáctico para $\text{id}+\text{id}*\text{id}$

4.2.6 Verificación del lenguaje generado por una gramática

Aunque los diseñadores de compiladores muy raras veces lo hacen para una gramática de lenguaje de programación completa, es útil poder razonar que un conjunto dado de producciones genera un lenguaje específico. Las construcciones problemáticas pueden estudiarse mediante la escritura de una gramática abstracta y concisa, y estudiando el lenguaje que genera. A continuación vamos a construir una gramática de este tipo, para instrucciones condicionales.

Una prueba de que una gramática G genera un lenguaje L consta de dos partes: mostrar que todas las cadenas generadas por G están en L y, de manera inversa, que todas las cadenas en L pueden generarse sin duda mediante G .

Ejemplo 4.12: Considere la siguiente gramática:

$$S \rightarrow (S) S \mid \epsilon \quad (4.13)$$

Tal vez no sea evidente desde un principio, pero esta gramática simple genera todas las cadenas de paréntesis balanceados, y sólo ese tipo de cadenas. Para ver por qué, primero mostraremos que todas las frases que se derivan de S son balanceadas, y después que todas las cadenas balanceadas se derivan de S . Para mostrar que todas las frases que pueden derivarse de S son balanceadas, utilizaremos una prueba inductiva sobre el número de pasos n en una derivación.

BASE: La base es $n = 1$. La única cadena de terminales que puede derivarse de S en un paso es la cadena vacía, que sin duda está balanceada.

INDUCCIÓN: Ahora suponga que todas las derivaciones de menos de n pasos producen frases balanceadas, y considere una derivación por la izquierda, con n pasos exactamente. Dicha derivación debe ser de la siguiente forma:

$$S \xRightarrow{lm} (S)S \xRightarrow{lm}^* (x)S \xRightarrow{lm}^* (x)y$$

Las derivaciones de x y y que provienen de S requieren menos de n pasos, por lo que en base a la hipótesis inductiva, x y y están balanceadas. Por lo tanto, la cadena $(x)y$ debe ser balanceada. Es decir, tiene un número equivalente de paréntesis izquierdos y derechos, y cada prefijo tiene, por lo menos, la misma cantidad de paréntesis izquierdos que derechos.

Habiendo demostrado entonces que cualquier cadena que se deriva de S está balanceada, debemos ahora mostrar que todas las cadenas balanceadas se derivan de S . Para ello, utilizaremos la inducción sobre la longitud de una cadena.

BASE: Si la cadena es de longitud 0, debe ser ϵ , la cual está balanceada.

INDUCCIÓN: Primero, observe que todas las cadenas balanceadas tienen longitud uniforme. Suponga que todas las cadenas balanceadas de una longitud menor a $2n$ se derivan de S , y considere una cadena balanceada w de longitud $2n$, $n \geq 1$. Sin duda, w empieza con un paréntesis izquierdo. Hagamos que (x) sea el prefijo no vacío más corto de w , que tenga el mismo número de paréntesis izquierdos y derechos. Así, w puede escribirse como $w = (x)y$, en donde x y y están balanceadas. Como x y y tienen una longitud menor a $2n$, pueden derivarse de S mediante la hipótesis inductiva. Por ende, podemos buscar una derivación de la siguiente forma:

$$S \Rightarrow (S)S \xRightarrow{*} (x)S \xRightarrow{*} (x)y$$

con lo cual demostramos que $w = (x)y$ también puede derivarse de S . $\square$

4.2.7 Comparación entre gramáticas libres de contexto y expresiones regulares

Antes de dejar esta sección sobre las gramáticas y sus propiedades, establecemos que las gramáticas son una notación más poderosa que las expresiones regulares. Cada construcción que puede describirse mediante una expresión regular puede describirse mediante una gramática, pero no al revés. De manera alternativa, cada lenguaje regular es un lenguaje libre de contexto, pero no al revés.

Por ejemplo, la expresión regular $(a|b)^*abb$ y la siguiente gramática:

$$\begin{aligned} A_0 &\rightarrow aA_0 \mid bA_0 \mid aA_1 \\ A_1 &\rightarrow bA_2 \\ A_2 &\rightarrow bA_3 \\ A_3 &\rightarrow \epsilon \end{aligned}$$

describen el mismo lenguaje, el conjunto de cadenas de as y bs que terminan en abb .

Podemos construir de manera mecánica una gramática para reconocer el mismo lenguaje que un autómata finito no determinista (AFN). La gramática anterior se construyó a partir del AFN de la figura 3.24, mediante la siguiente construcción:

1. Para cada estado i del AFN, crear un no terminal A_i .
2. Si el estado i tiene una transición al estado j con la entrada a , agregar la producción $A_i \rightarrow aA_j$. Si el estado i pasa al estado j con la entrada ϵ , agregar la producción $A_i \rightarrow A_j$.
3. Si i es un estado de aceptación, agregar $A_i \rightarrow \epsilon$.
4. Si i es el estado inicial, hacer que A_i sea el símbolo inicial de la gramática.

Por otra parte, el lenguaje $L = \{a^n b^n \mid n \geq 1\}$ con un número equivalente de as y bs es un ejemplo de prototipo de un lenguaje que puede describirse mediante una gramática, pero no mediante una expresión regular. Para ver por qué, suponga que L es el lenguaje definido por alguna expresión regular. Construiríamos un AFD D con un número finito de estados, por decir k , para aceptar a L . Como D sólo tiene k estados, para una entrada que empieza con más de k as , D debe entrar a cierto estado dos veces, por decir s_i , como en la figura 4.6. Suponga que la ruta de s_i de vuelta a sí mismo se etiqueta con una secuencia a^{j-i} . Como $a^i b^i$ está en el lenguaje, debe haber una ruta etiquetada como b^i desde s_i hasta un estado de aceptación f . Pero, entonces también hay una ruta que sale desde el estado s_0 y pasa a través de s_i para llegar a f , etiquetada como $a^i b^i$, como se muestra en la figura 4.6. Por ende, D también acepta a $a^j b^i$, que no está en el lenguaje, lo cual contradice la suposición de que L es el lenguaje aceptado por D .

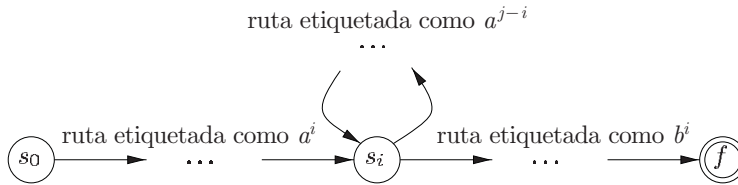


Figura 4.6: Un AFD D que acepta a $a^i b^i$ y a $a^j b^i$

En lenguaje coloquial, decimos que “los autómatas finitos no pueden contar”, lo cual significa que un autómata finito no puede aceptar un lenguaje como $\{a^n b^n \mid n \geq 1\}$, que requiera que el autómata lleve la cuenta del número de as antes de ver las bs . De igual forma, “una gramática puede contar dos elementos pero no tres”, como veremos cuando hablemos sobre las construcciones de lenguajes que no son libres de contexto en la sección 4.3.5.

4.2.8 Ejercicios para la sección 4.2

Ejercicio 4.2.1: Considere la siguiente gramática libre de contexto:

$$S \rightarrow S S + \mid S S * \mid a$$

y la cadena $aa + a*$.

- Proporcione una derivación por la izquierda para la cadena.
- Proporcione una derivación por la derecha para la cadena.
- Proporcione un árbol de análisis sintáctico para la cadena.

! d) ¿La gramática es ambigua o no? Justifique su respuesta.

! e) Describa el lenguaje generado por esta gramática.

Ejercicio 4.2.2: Repita el ejercicio 4.2.1 para cada una de las siguientes gramáticas y cadenas:

- a) $S \rightarrow 0 S 1 \mid 0 1$ con la cadena 000111.
- b) $S \rightarrow + S S \mid * S S \mid a$ con la cadena $+ * aaa$.
- ! c) $S \rightarrow S (S) S \mid \epsilon$ con la cadena $((()))$.
- ! d) $S \rightarrow S + S \mid S S \mid (S) \mid S * \mid a$ con la cadena $(a + a) * a$.
- ! e) $S \rightarrow (L) \mid a$ y $L \rightarrow L, S \mid S$ con la cadena $((a, a), a, (a))$.
- !! f) $S \rightarrow a S b S \mid b S a S \mid \epsilon$ con la cadena $aabbab$.
- ! g) La siguiente gramática para las expresiones booleanas:

$$\begin{aligned} bexpr &\rightarrow bexpr \textbf{ or } bterm \mid bterm \\ bterm &\rightarrow bterm \textbf{ and } bfactor \mid bfactor \\ bfactor &\rightarrow \textbf{ not } bfactor \mid (bexpr) \mid \textbf{ true } \mid \textbf{ false } \end{aligned}$$

Ejercicio 4.2.3: Diseñe gramáticas para los siguientes lenguajes:

- a) El conjunto de todas las cadenas de 0s y 1s, de tal forma que justo antes de cada 0 vaya por lo menos un 1.
- ! b) El conjunto de todas las cadenas de 0s y 1s que sean *palíndromos*; es decir, que la cadena se lea igual al derecho y al revés.
- ! c) El conjunto de todas las cadenas de 0s y 1s con un número igual de 0s y 1s.
- !! d) El conjunto de todas las cadenas de 0s y 1s con un número desigual de 0s y 1s.
- ! e) El conjunto de todas las cadenas de 0s y 1s en donde 011 no aparece como una subcadena.
- !! f) El conjunto de todas las cadenas de 0s y 1s de la forma xy , en donde $x \neq y$, y x y y tienen la misma longitud.

! Ejercicio 4.2.4: Hay una notación de gramática extendida de uso común. En esta notación, los corchetes y las llaves en los cuerpos de las producciones son meta símbolos (como $\rightarrow$ o $\mid$) con los siguientes significados:

- i) Los corchetes alrededor de un símbolo o símbolos gramaticales denota que estas construcciones son opcionales. Por ende, la producción $A \rightarrow X [Y] Z$ tiene el mismo efecto que las dos producciones $A \rightarrow X Y Z$ y $A \rightarrow X Z$.
- ii) Las llaves alrededor de un símbolo o símbolos gramaticales indican que estos símbolos pueden repetirse cualquier número de veces, incluyendo cero. Por ende, $A \rightarrow X \{Y Z\}$ tiene el mismo efecto que la secuencia infinita de producciones $A \rightarrow X$, $A \rightarrow X Y Z$, $A \rightarrow X Y Z Y Z$, y así sucesivamente.

Muestre que estas dos extensiones no agregan potencia a las gramáticas; es decir, cualquier lenguaje que pueda generarse mediante una gramática con estas extensiones, podrá generarse mediante una gramática sin las extensiones.

Ejercicio 4.2.5: Use las llaves descritas en el ejercicio 4.2.4 para simplificar la siguiente gramática para los bloques de instrucciones y las instrucciones condicionales:

$$\begin{array}{ll} instr & \rightarrow \text{if } expr \text{ then } instr \text{ else } instr \\ & | \text{if } instr \text{ then } instr \\ & | \text{begin } listaInstr \text{ end} \\ listaInstr & \rightarrow instr ; listaInstr \mid instr \end{array}$$

! Ejercicio 4.2.6: Extienda la idea del ejercicio 4.2.4 para permitir cualquier expresión regular de símbolos gramaticales en el cuerpo de una producción. Muestre que esta extensión no permite que las gramáticas definan nuevos lenguajes.

! Ejercicio 4.2.7: Un símbolo gramatical X (terminal o no terminal) es *inútil* si no hay derivación de la forma $S \xRightarrow{*} wXy \xRightarrow{*} wxy$. Es decir, X nunca podrá aparecer en la derivación de un enunciado.

- Proporcione un algoritmo para eliminar de una gramática todas las producciones que contengan símbolos inútiles.
- Aplice su algoritmo a la siguiente gramática:

$$\begin{array}{ll} S & \rightarrow 0 \mid A \\ A & \rightarrow AB \\ B & \rightarrow 1 \end{array}$$

Ejercicio 4.2.8: La gramática en la figura 4.7 genera declaraciones para un solo identificador numérico; estas declaraciones involucran a cuatro propiedades distintas e independientes de números.

$$\begin{array}{ll} instr & \rightarrow \text{declare id } listaOpciones \\ listaOpciones & \rightarrow listaOpciones opcion \mid \epsilon \\ opcion & \rightarrow modo \mid escala \mid precision \mid base \\ modo & \rightarrow \text{real} \mid \text{complex} \\ escala & \rightarrow \text{fixed} \mid \text{floating} \\ precision & \rightarrow \text{single} \mid \text{double} \\ base & \rightarrow \text{binary} \mid \text{decimal} \end{array}$$

Figura 4.7: Una gramática para declaraciones con varios atributos

- Generalice la gramática de la figura 4.7, permitiendo n opciones A_i , para algunas n fijas y para $i = 1, 2, \dots, n$, en donde A_i puede ser a_i o b_i . Su gramática deberá usar sólo $O(n)$ símbolos gramaticales y tener una longitud total de producciones igual a $O(n)$.

- ! b) La gramática de la figura 4.7 y su generalización en la parte (a) permite declaraciones que son contradictorias o redundantes, tales como:

```
declare foo real fixed real floating
```

Podríamos insistir en que la sintaxis del lenguaje prohíbe dichas declaraciones; es decir, cada declaración generada por la gramática tiene exactamente un valor para cada una de las n opciones. Si lo hacemos, entonces para cualquier n fija hay sólo un número finito de declaraciones legales. Por ende, el lenguaje de declaraciones legales tiene una gramática (y también una expresión regular), al igual que cualquier lenguaje finito. La gramática obvia, en la cual el símbolo inicial tiene una producción para cada declaración legal, tiene $n!$ producciones y una longitud de producciones total de $O(n \times n!)$. Hay que esforzarse más: una longitud de producciones total que sea $O(n2^n)$.

- !! c) Muestre que cualquier gramática para la parte (b) debe tener una longitud de producciones total de por lo menos 2^n .
- d) ¿Qué dice la parte (c) acerca de la viabilidad de imponer la no redundancia y la no contradicción entre las opciones en las declaraciones, a través de la sintaxis del lenguaje de programación?

4.3 Escritura de una gramática

Las gramáticas son capaces de describir casi la mayoría de la sintaxis de los lenguajes de programación. Por ejemplo, el requerimiento de que los identificadores deben declararse antes de usarse, no puede describirse mediante una gramática libre de contexto. Por lo tanto, las secuencias de los tokens que acepta un analizador sintáctico forman un superconjunto del lenguaje de programación; las fases siguientes del compilador deben analizar la salida del analizador sintáctico, para asegurar que cumpla con las reglas que no verifica el analizador sintáctico.

Esta sección empieza con una discusión acerca de cómo dividir el trabajo entre un analizador léxico y un analizador sintáctico. Después consideraremos varias transformaciones que podrían aplicarse para obtener una gramática más adecuada para el análisis sintáctico. Una técnica puede eliminar la ambigüedad en la gramática, y las otras (eliminación de recursividad por la izquierda y factorización por la izquierda) son útiles para rescribir las gramáticas, de manera que sean adecuadas para el análisis sintáctico descendente. Concluiremos esta sección considerando algunas construcciones de los lenguajes de programación que ninguna gramática puede describir.

4.3.1 Comparación entre análisis léxico y análisis sintáctico

Como observamos en la sección 4.2.7, todo lo que puede describirse mediante una expresión regular también puede describirse mediante una gramática. Por lo tanto, sería razonable preguntar: “¿Por qué usar expresiones regulares para definir la sintaxis léxica de un lenguaje?” Existen varias razones.

1. Al separar la estructura sintáctica de un lenguaje en partes léxicas y no léxicas, se proporciona una manera conveniente de colocar en módulos la interfaz de usuario de un compilador en dos componentes de un tamaño manejable.
2. Las reglas léxicas de un lenguaje son con frecuencia bastante simples, y para describirlas no necesitamos una notación tan poderosa como las gramáticas.
3. Por lo general, las expresiones regulares proporcionan una notación más concisa y fácil de entender para los tokens, en comparación con las gramáticas.
4. Pueden construirse analizadores léxicos más eficientes en forma automática a partir de expresiones regulares, en comparación con las gramáticas arbitrarias.

No hay lineamientos firmes en lo que se debe poner en las reglas léxicas, en contraste a las reglas sintácticas. Las expresiones regulares son muy útiles para describir la estructura de las construcciones como los identificadores, las constantes, las palabras reservadas y el espacio en blanco. Por otro lado, las gramáticas son muy útiles para describir estructuras anidadas, como los paréntesis balanceados, las instrucciones begin-end relacionadas, las instrucciones if-then-else correspondientes, etcétera. Estas estructuras anidadas no pueden describirse mediante las expresiones regulares.

4.3.2 Eliminación de la ambigüedad

Algunas veces, una gramática ambigua puede describirse para eliminar la ambigüedad. Como ejemplo, vamos a eliminar la ambigüedad de la siguiente gramática del “else colgante”:

$$\begin{array}{lcl}
 instr & \rightarrow & \text{if } expr \text{ then } instr \\
 & & | \text{ if } expr \text{ then } instr \text{ else } instr \\
 & & | \text{ otra}
 \end{array} \tag{4.14}$$

Aquí, “otra” representa a cualquier otra instrucción. De acuerdo con esta gramática, la siguiente instrucción condicional compuesta:

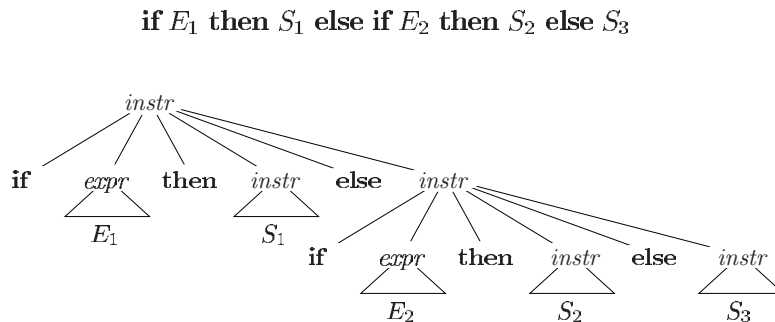


Figura 4.8: Árbol de análisis sintáctico para una instrucción condicional

tiene el árbol de análisis sintáctico que se muestra en la figura 4.8.¹ La gramática (4.14) es ambigua, ya que la cadena

$$\mathbf{if } E_1 \mathbf{ then if } E_2 \mathbf{ then } S_1 \mathbf{ else } S_2 \quad (4.15)$$

tiene los dos árboles de análisis sintáctico que se muestran en la figura 4.9.

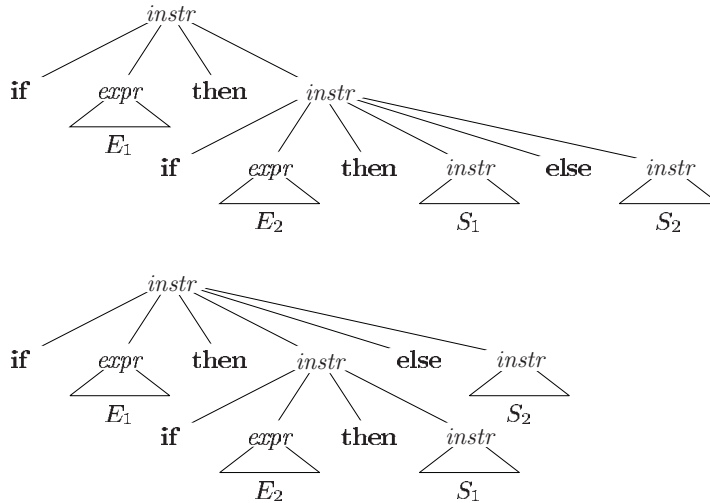


Figura 4.9: Dos árboles de análisis sintáctico para un enunciado ambiguo

En todos los lenguajes de programación con instrucciones condicionales de esta forma, se prefiere el primer árbol de análisis sintáctico. La regla general es, “Relacionar cada **else** con el **then** más cercano que no esté relacionado”.² Esta regla para eliminar ambigüedad puede, en teoría, incorporarse directamente en una gramática, pero en la práctica raras veces se integra a las producciones.

Ejemplo 4.16: Podemos describir la gramática del **else** colgante (4.14) como la siguiente gramática sin ambigüedades. La idea es que una instrucción que aparece entre un **then** y un **else** debe estar “relacionada”; es decir, la instrucción interior no debe terminar con un **then** sin relacionar o abierto. Una instrucción relacionada es una instrucción **if-then-else** que no contiene instrucciones abiertas, o es cualquier otro tipo de instrucción incondicional. Por ende, podemos usar la gramática de la figura 4.10. Esta gramática genera las mismas cadenas que la gramática del **else** colgante (4.14), pero sólo permite un análisis sintáctico para la cadena (4.15); en específico, el que asocia a cada **else** con la instrucción **then** más cercana que no haya estado relacionada antes. □

¹Los subíndices en *E* y *S* son sólo para diferenciar las distintas ocurrencias del mismo no terminal, por lo cual no implican no terminales distintos.

²Hay que tener en cuenta que *C* y sus derivados se incluyen en esta clase. Aun cuando la familia de lenguajes *C* no utiliza la palabra clave **then**, su función se representa mediante el paréntesis de cierre para la condición que va después de **if**.

$$\begin{array}{ll}
instr & \rightarrow \quad instr_relacionada \\
& \quad | \quad instr_abierta \\
instr_relacionada & \rightarrow \quad \textbf{if } expr \textbf{ then } instr_relacionada \textbf{ else } instr_relacionada \\
& \quad | \quad \textbf{otra} \\
instr_abierta & \rightarrow \quad \textbf{if } expr \textbf{ then } instr \\
& \quad | \quad \textbf{if } expr \textbf{ then } instr_relacionada \textbf{ else } instr_abierta
\end{array}$$

Figura 4.10: Gramática sin ambigüedades para las instrucciones if-then-else

4.3.3 Eliminación de la recursividad por la izquierda

Una gramática es *recursiva por la izquierda* si tiene una terminal A tal que haya una derivación $A \xRightarrow{\pm} A\alpha$ para cierta cadena α . Los métodos de análisis sintáctico descendentes no pueden manejar las gramáticas recursivas por la izquierda, por lo que se necesita una transformación para eliminar la recursividad por la izquierda. En la sección 2.4.5 hablamos sobre la *recursividad inmediata por la izquierda*, en donde hay una producción de la forma $A \rightarrow A\alpha$. Aquí estudiaremos el caso general. En la sección 2.4.5, mostramos cómo el par recursivo por la izquierda de producciones $A \rightarrow A\alpha \mid \beta$ podía sustituirse mediante las siguientes producciones no recursivas por la izquierda:

$$\begin{array}{l}
A \rightarrow \beta A' \\
A' \rightarrow \alpha A' \mid \epsilon
\end{array}$$

sin cambiar las cadenas que se derivan de A . Esta regla por sí sola basta para muchas gramáticas.

Ejemplo 4.17: La gramática de expresiones no recursivas por la izquierda (4.2), que se repite a continuación:

$$\begin{array}{l}
E \rightarrow T E' \\
E' \rightarrow + T E' \\
T \rightarrow F T' \\
T' \rightarrow * F T' \\
F \rightarrow (E) \mid \textbf{id}
\end{array}$$

se obtiene mediante la eliminación de la recursividad inmediata por la izquierda de la gramática de expresiones (4.1). El par recursivo por la izquierda de las producciones $E \rightarrow E + T \mid T$ se sustituye mediante $E \rightarrow T E'$ y $E' \rightarrow + T E' \mid \epsilon$. Las nuevas producciones para T y T' se obtienen de manera similar, eliminando la recursividad inmediata por la izquierda. $\square$

La recursividad inmediata por la izquierda puede eliminarse mediante la siguiente técnica, que funciona para cualquier número de producciones A . En primer lugar, se agrupan las producciones de la siguiente manera:

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

en donde ninguna β_i termina con una A . Después, se sustituyen las producciones A mediante lo siguiente:

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A' \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon \end{aligned}$$

El no terminal A genera las mismas cadenas que antes, pero ya no es recursiva por la izquierda. Este procedimiento elimina toda la recursividad por la izquierda de las producciones A y A' (siempre y cuando ninguna α_i sea ϵ), pero no elimina la recursividad por la izquierda que incluye a las derivaciones de dos o más pasos. Por ejemplo, considere la siguiente gramática:

$$\begin{aligned} S &\rightarrow A a \mid b \\ A &\rightarrow A c \mid S d \mid \epsilon \end{aligned} \tag{4.18}$$

El no terminal S es recursiva por la izquierda, ya que $S \Rightarrow Aa \Rightarrow Sda$, pero no es inmediatamente recursiva por la izquierda.

El Algoritmo 4.19, que se muestra a continuación, elimina en forma sistemática la recursividad por la izquierda de una gramática. Se garantiza que funciona si la gramática no tiene ciclos (derivaciones de la forma $A \xRightarrow{+} A$) o producciones ϵ (producciones de la forma $A \rightarrow \epsilon$). Los ciclos pueden eliminarse en forma sistemática de una gramática, al igual que las producciones ϵ (vea los ejercicios 4.4.6 y 4.4.7).

Algoritmo 4.19: Eliminación de la recursividad por la izquierda.

ENTRADA: La gramática G sin ciclos ni producciones ϵ .

SALIDA: Una gramática equivalente sin recursividad por la izquierda.

MÉTODO: Aplicar el algoritmo de la figura 4.11 a G . Observe que la gramática no recursiva por la izquierda resultante puede tener producciones ϵ . $\square$

- 1) ordenar los no terminales de cierta forma $A_1, A_2, \dots, A_n$.
- 2) **for** (cada i de 1 a n) {
- 3) **for** (cada j de 1 a $i - 1$) {
- 4) sustituir cada producción de la forma $A_i \rightarrow A_j \gamma$ por las
 producciones $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \cdots \mid \delta_k \gamma$, en donde
 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \cdots \mid \delta_k$ sean todas producciones A_j actuales
- 5) }
- 6) eliminar la recursividad inmediata por la izquierda entre las producciones A_i
- 7) }

Figura 4.11: Algoritmo para eliminar la recursividad por la izquierda de una gramática

El procedimiento en la figura 4.11 funciona de la siguiente manera. En la primera iteración para $i = 1$, el ciclo for externo de las líneas (2) a la (7) elimina cualquier recursividad inmediata por la izquierda entre las producciones A_1 . Cualquier producción A_1 restante de la forma $A_1 \rightarrow A_l \alpha$ debe, por lo tanto, tener $l > 1$. Después de la i -ésima iteración del ciclo for externo, todas las no terminales A_k , en donde $k < i$, se “limpian”; es decir, cualquier producción $A_k \rightarrow A_l \alpha$ debe tener $l > k$. Como resultado, en la i -ésima iteración, el ciclo interno de las líneas (3) a la (5) eleva en forma progresiva el límite inferior en cualquier producción $A_i \rightarrow A_m \alpha$, hasta tener $m \geq i$.

Después, la eliminación de la recursividad inmediata por la izquierda para las producciones A_i en la línea (6) obliga a que m sea mayor que i .

Ejemplo 4.20: Vamos a aplicar el Algoritmo 4.19 a la gramática (4.18). Técnicamente, no se garantiza que el algoritmo vaya a funcionar debido a la producción ϵ , pero en este caso, la producción $A \rightarrow \epsilon$ resulta ser inofensiva.

Ordenamos los no terminales S, A . No hay recursividad inmediata por la izquierda entre las producciones S , por lo que no ocurre nada durante el ciclo externo para $i = 1$. Para $i = 2$, sustituimos la S en $A \rightarrow S d$ para obtener las siguientes producciones A .

$$A \rightarrow A c \mid A a d \mid b d \mid \epsilon$$

Al eliminar la recursividad inmediata por la izquierda entre las producciones A produce la siguiente gramática:

$$\begin{aligned} S &\rightarrow A a \mid b \\ A &\rightarrow b d A' \mid A' \\ A' &\rightarrow c A' \mid a d A' \mid \epsilon \end{aligned}$$

□

4.3.4 Factorización por la izquierda

La factorización por la izquierda es una transformación gramatical, útil para producir una gramática adecuada para el análisis sintáctico predictivo, o descendente. Cuando la elección entre dos producciones A alternativas no está clara, tal vez podamos describir las producciones para diferir la decisión hasta haber visto la suficiente entrada como para poder realizar la elección correcta.

Por ejemplo, si tenemos las siguientes dos producciones:

$$\begin{aligned} instr &\rightarrow \textbf{if } expr \textbf{ then } instr \textbf{ else } instr \\ &\mid \textbf{if } expr \textbf{ then } instr \end{aligned}$$

al ver la entrada **if**, no podemos saber de inmediato qué producción elegir para expandir $instr$. En general, si $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ son dos producciones A , y la entrada empieza con una cadena no vacía derivada de α , no sabemos si debemos expandir A a $\alpha\beta_1$ o a $\alpha\beta_2$. No obstante, podemos diferir la decisión si expandimos A a $\alpha A'$. Así, después de ver la entrada derivada de α , expandimos A' a β_1 o a β_2 . Es decir, si se factorizan por la izquierda, las producciones originales se convierten en:

$$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta_1 \mid \beta_2 \end{aligned}$$

Algoritmo 4.21: Factorización por la izquierda de una gramática.

ENTRADA: La gramática G .

SALIDA: Una gramática equivalente factorizada por la izquierda.

MÉTODO: Para cada no terminal A , encontrar el prefijo α más largo que sea común para una o más de sus alternativas. Si $\alpha \neq \epsilon$ (es decir, si hay un prefijo común no trivial), se sustituyen todas las producciones A , $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \cdots \mid \alpha\beta_n \mid \gamma$, en donde γ representa a todas las alternativas que no empiezan con α , mediante lo siguiente:

$$\begin{aligned} A &\rightarrow \alpha A' \mid \gamma \\ A' &\rightarrow \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n \end{aligned}$$

Aquí, A' es un no terminal nuevo. Se aplica esta transformación en forma repetida hasta que no haya dos alternativas para un no terminal que tengan un prefijo común. $\square$

Ejemplo 4.22: La siguiente gramática abstrae el problema del “else colgante”:

$$\begin{aligned} S &\rightarrow i E t S \mid i E t S e S \mid a \\ E &\rightarrow b \end{aligned} \tag{4.23}$$

Aquí, i , t y e representan a **if**, **then** y **else**; E y S representan “expresión condicional” e “instrucción”. Si se factoriza a la izquierda, esta gramática se convierte en:

$$\begin{aligned} S &\rightarrow i E t S S' \mid a \\ S' &\rightarrow e S \mid \epsilon \\ E &\rightarrow b \end{aligned} \tag{4.24}$$

Así, podemos expandir S a $iEtSS'$ con la entrada i , y esperar hasta que se haya visto $iEtS$ para decidir si se va a expandir S' a eS o a ϵ . Desde luego que estas gramáticas son ambiguas, y con la entrada e no quedará claro qué alternativa debe elegirse para S' . El ejemplo 4.33 habla sobre cómo salir de este dilema. $\square$

4.3.5 Construcciones de lenguajes que no son libres de contexto

Algunas construcciones sintácticas que se encuentran en los lenguajes de programación ordinarios no pueden especificarse sólo mediante el uso de gramáticas. Aquí consideraremos dos de estas construcciones, usando lenguajes abstractos simples para ilustrar las dificultades.

Ejemplo 4.25: El lenguaje en este ejemplo abstrae el problema de comprobar que se declaren los identificadores antes de poder usarlos en un programa. El lenguaje consiste en cadenas de la forma wcw , en donde la primera w representa la declaración de un identificador w , c representa un fragmento intermedio del programa, y la segunda w representa el uso del identificador.

El lenguaje abstracto es $L_1 = \{wcw \mid w \text{ está en } (\mathbf{a|b})^*\}$. L_1 consiste en todas las palabras compuestas de una cadena repetida de as y bs separadas por c , como $aabcaab$. Aunque no lo vamos a demostrar aquí, la característica de no ser libre de contexto de L_1 implica directamente que los lenguajes de programación como C y Java no sean libres de contexto, los cuales requieren la declaración de los identificadores antes de usarlos, además de permitir identificadores de longitud arbitraria.

Por esta razón, una gramática para C o Java no hace diferencias entre los identificadores que son cadenas distintas de caracteres. En vez de ello, todos los identificadores se representan

mediante un token como **id** en la gramática. En un compilador para dicho lenguaje, la fase de análisis semántico comprueba que los identificadores se declaren antes de usarse. $\square$

Ejemplo 4.26: El lenguaje, que no es independiente del contexto en este ejemplo, abstrae el problema de comprobar que el número de parámetros formales en la declaración de una función coincida con el número de parámetros actuales en un uso de la función. El lenguaje consiste en cadenas de la forma $a^n b^m c^n d^m$. (Recuerde que a^n significa a escrita n veces). Aquí, a^n y b^m podrían representar las listas de parámetros formales de dos funciones declaradas para tener n y m argumentos, respectivamente, mientras que c^n y d^m representan las listas de los parámetros actuales en las llamadas a estas dos funciones.

El lenguaje abstracto es $L_2 = \{a^n b^m c^n d^m \mid n \geq 1 \text{ y } m \geq 1\}$. Es decir, L_2 consiste de cadenas en el lenguaje generado por la expresión regular $\mathbf{a^*b^*c^*d^*}$, de tal forma que el número de a s y c s y de b s y d s sea igual. Este lenguaje no es independiente del contexto.

De nuevo, la sintaxis común de las declaraciones de las funciones y los usos no se responsabiliza de contar el número de parámetros. Por ejemplo, la llamada a una función en un lenguaje similar a C podría especificarse de la siguiente manera:

$$\begin{array}{ll} instr & \rightarrow \mathbf{id} \text{ (lista_expr)} \\ lista_expr & \rightarrow lista_expr, \text{ expr} \\ & | \text{ expr} \end{array}$$

con producciones adecuadas para $expr$. La comprobación de que el número de parámetros en una llamada sea correcto se realiza por lo general durante la fase del análisis semántico. $\square$

4.3.6 Ejercicios para la sección 4.3

Ejercicio 4.3.1: La siguiente gramática es para expresiones regulares sobre los símbolos a y b solamente, usando $+$ en vez de $|$ para la unión, con lo cual se evita el conflicto con el uso de la barra vertical como un meta símbolo en las gramáticas:

$$\begin{array}{ll} rexr & \rightarrow rexr + rterm \mid rterm \\ rterm & \rightarrow rterm rfactor \mid rfactor \\ rfactor & \rightarrow rfactor * \mid rprimario \\ rprimario & \rightarrow \mathbf{a} \mid \mathbf{b} \end{array}$$

- Factorice esta gramática por la izquierda.
- ¿La factorización por la izquierda hace a la gramática adecuada para el análisis sintáctico descendente?
- Además de la factorización por la izquierda, elimine la recursividad por la izquierda de la gramática original.
- ¿La gramática resultante es adecuada para el análisis sintáctico descendente?

Ejercicio 4.3.2: Repita el ejercicio 4.3.1 con las siguientes gramáticas:

- La gramática del ejercicio 4.2.1.
- La gramática del ejercicio 4.2.2(a).

- c) La gramática del ejercicio 4.2.2(c).
- d) La gramática del ejercicio 4.2.2(e).
- e) La gramática del ejercicio 4.2.2(g).

! Ejercicio 4.3.3: Se propone la siguiente gramática para eliminar la “ambigüedad del else colgante”, descrita en la sección 4.3.2:

$$\begin{array}{ll}
 instr & \rightarrow \text{if } expr \text{ then } instr \\
 & \quad | \quad instrRelacionada \\
 instrRelacionada & \rightarrow \text{if } expr \text{ then } instrRelacionada \text{ else } instr \\
 & \quad | \quad otra
 \end{array}$$

Muestre que esta gramática sigue siendo ambigua.

4.4 Análisis sintáctico descendente

El análisis sintáctico descendente puede verse como el problema de construir un árbol de análisis sintáctico para la cadena de entrada, partiendo desde la raíz y creando los nodos del árbol de análisis sintáctico en preorden (profundidad primero, como vimos en la sección 2.3.4). De manera equivalente, podemos considerar el análisis sintáctico descendente como la búsqueda de una derivación por la izquierda para una cadena de entrada.

Ejemplo 4.27: La secuencia de árboles de análisis sintáctico en la figura 4.12 para la entrada **id+id*id** es un análisis sintáctico descendente, de acuerdo con la gramática (4.2), que repetimos a continuación:

$$\begin{array}{ll}
 E & \rightarrow T E' \\
 E' & \rightarrow + T E' \mid \epsilon \\
 T & \rightarrow F T' \\
 T' & \rightarrow * F T' \mid \epsilon \\
 F & \rightarrow (E) \mid \text{id}
 \end{array} \tag{4.28}$$

Esta secuencia de árboles corresponde a una derivación por la izquierda de la entrada. $\square$

En cada paso de un análisis sintáctico descendente, el problema clave es el de determinar la producción que debe aplicarse para un no terminal, por decir A . Una vez que se elige una producción A , el resto del proceso de análisis sintáctico consiste en “relacionar” los símbolos terminales en el cuerpo de la producción con la cadena de entrada.

Esta sección empieza con una forma general del análisis sintáctico descendente, conocida como análisis sintáctico de descenso recursivo, la cual puede requerir de un rastreo hacia atrás para encontrar la producción A correcta que debe aplicarse. La sección 2.4.2 introdujo el análisis sintáctico predictivo, un caso especial de análisis sintáctico de descenso recursivo, en donde no se requiere un rastreo hacia atrás. El análisis sintáctico predictivo elige la producción A correcta mediante un análisis por adelantado de la entrada, en donde se ve un número fijo de símbolos adelantados; por lo general, sólo necesitamos ver un símbolo por adelantado (es decir, el siguiente símbolo de entrada).

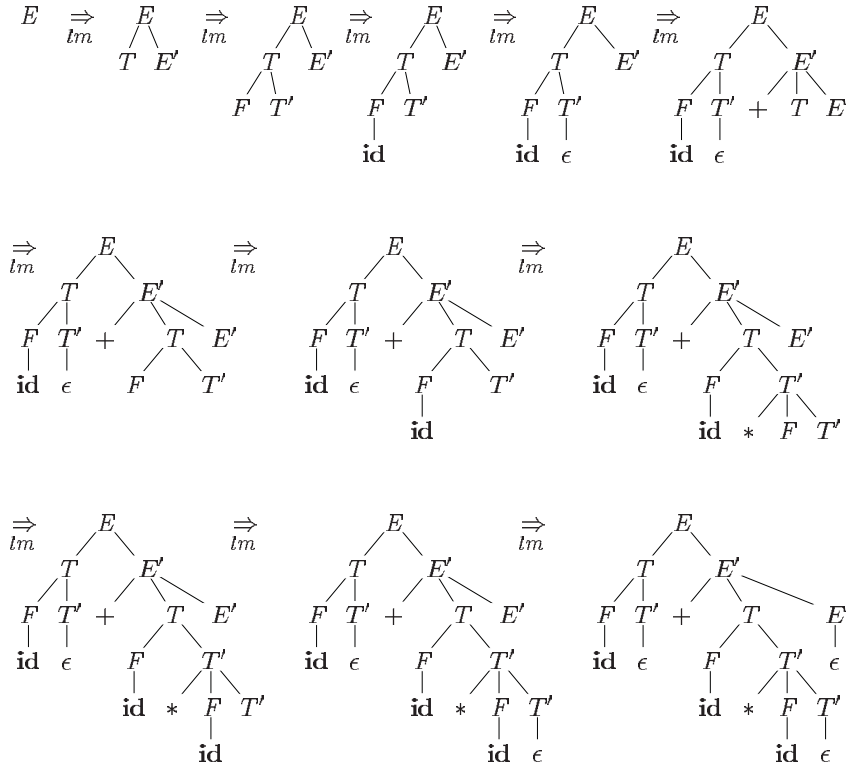


Figura 4.12: Análisis sintáctico descendente para $\text{id} + \text{id} * \text{id}$

Por ejemplo, considere el análisis sintáctico descendente en la figura 4.12, en la cual se construye un árbol con dos nodos etiquetados como E' . En el primer nodo E' (en preorden), se elige la producción $E' \rightarrow +TE'$; en el segundo nodo E' , se elige la producción $E' \rightarrow \epsilon$. Un analizador sintáctico predictivo puede elegir una de las producciones E' mediante el análisis del siguiente símbolo de entrada.

A la clase de gramáticas para las cuales podemos construir analizadores sintácticos predictivos que analicen k símbolos por adelantado en la entrada, se le conoce algunas veces como la clase $LL(k)$. En la sección 4.4.3 hablaremos sobre la clase $LL(1)$, pero presentaremos antes ciertos cálculos, llamados PRIMERO y SIGUIENTE, en la sección 4.4.2. A partir de los conjuntos PRIMERO y SIGUIENTE para una gramática, construiremos “tablas de análisis sintáctico predictivo”, las cuales hacen explícita la elección de la producción durante el análisis sintáctico descendente. Estos conjuntos también son útiles durante el análisis sintáctico ascendente.

En la sección 4.4.4 proporcionaremos un algoritmo de análisis sintáctico no recursivo que mantiene una pila en forma explícita, en vez de hacerlo en forma implícita mediante llamadas recursivas. Por último, en la sección 4.4.5 hablaremos sobre la recuperación de errores durante el análisis sintáctico descendente.

4.4.1 Análisis sintáctico de descenso recursivo

```

void A() {
1)      Elegir una producción  $A$ ,  $A \rightarrow X_1X_2 \cdots X_k$ ;
2)      for (  $i = 1$  a  $k$  ) {
3)          if (  $X_i$  es un no terminal )
4)              llamar al procedimiento  $X_i()$ ;
5)          else if (  $X_i$  es igual al símbolo de entrada actual  $a$  )
6)              avanzar la entrada hasta el siguiente símbolo;
7)          else /* ha ocurrido un error */;
      }
}

```

Figura 4.13: Un procedimiento ordinario para un no terminal en un analizador sintáctico descendente

Un programa de análisis sintáctico de descenso recursivo consiste en un conjunto de procedimientos, uno para cada no terminal. La ejecución empieza con el procedimiento para el símbolo inicial, que se detiene y anuncia que tuvo éxito si el cuerpo de su procedimiento explora toda la cadena completa de entrada. En la figura 4.13 aparece el pseudocódigo para un no terminal común. Observe que este pseudocódigo es no determinista, ya que empieza eligiendo la producción A que debe aplicar de una forma no especificada.

El descenso recursivo general puede requerir de un rastreo hacia atrás; es decir, tal vez requiera exploraciones repetidas sobre la entrada. Sin embargo, raras veces se necesita el rastreo hacia atrás para analizar las construcciones de un lenguaje de programación, por lo que los analizadores sintácticos con éste no se ven con frecuencia. Incluso para situaciones como el análisis sintáctico de un lenguaje natural, el rastreo hacia atrás no es muy eficiente, por lo cual se prefieren métodos tabulares como el algoritmo de programación dinámica del ejercicio 4.4.9, o el método de Earley (vea las notas bibliográficas).

Para permitir el rastreo hacia atrás, hay que modificar el código de la figura 4.13. En primer lugar, no podemos elegir una producción A única en la línea (1), por lo que debemos probar cada una de las diversas producciones en cierto orden. Después, el fallo en la línea (7) no es definitivo, sino que sólo sugiere que necesitamos regresar a la línea (1) y probar otra producción A . Sólo si no hay más producciones A para probar es cuando declaramos que se ha encontrado un error en la entrada. Para poder probar otra producción A , debemos restablecer el apuntador de entrada a la posición en la que se encontraba cuando llegamos por primera vez a la línea (1). Es decir, se requiere una variable local para almacenar este apuntador de entrada, para un uso futuro.

Ejemplo 4.29: Considere la siguiente gramática:

$$\begin{array}{lcl}
 S & \rightarrow & c A d \\
 A & \rightarrow & a b \mid a
 \end{array}$$

Para construir un árbol de análisis sintáctico descendente para la cadena de entrada $w = cad$, empezamos con un árbol que consiste en un solo nodo etiquetado como S , y el apuntador de entrada apunta a c , el primer símbolo de w . S sólo tiene una producción, por lo que la utilizamos

para expandir S y obtener el árbol de la figura 4.14(a). La hoja por la izquierda, etiquetada como c , coincide con el primer símbolo de la entrada w , por lo que avanzamos el apuntador de entrada hasta a , el segundo símbolo de w , y consideramos la siguiente hoja, etiquetada como A .

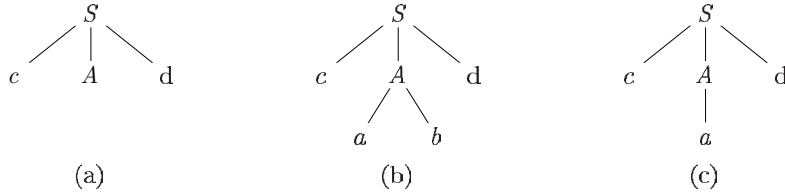


Figura 4.14: Los pasos de un análisis sintáctico descendente

Ahora expandimos A mediante la primera alternativa $A \rightarrow a b$ para obtener el árbol de la figura 4.14(b). Tenemos una coincidencia para el segundo símbolo de entrada a , por lo que avanzamos el apuntador de entrada hasta d , el tercer símbolo de entrada, y comparamos a d con la siguiente hoja, etiquetada con b . Como b no coincide con d , reportamos un error y regresamos a A para ver si hay otra alternativa para A que no hayamos probado y que pueda producir una coincidencia.

Al regresar a A , debemos restablecer el apuntador de entrada a la posición 2, la posición que tenía cuando llegamos por primera vez a A , lo cual significa que el procedimiento para A debe almacenar el apuntador de entrada en una variable local.

La segunda alternativa para A produce el árbol de la figura 4.14(c). La hoja a coincide con el segundo símbolo de w y la hoja d coincide con el tercer símbolo. Como hemos producido un árbol de análisis sintáctico para w , nos detenemos y anunciamos que se completó el análisis sintáctico con éxito. $\square$

Una gramática recursiva por la izquierda puede hacer que un analizador sintáctico de descenso recursivo, incluso uno con rastreo hacia atrás, entre en un ciclo infinito. Es decir, al tratar de expandir una no terminal A , podríamos en un momento dado encontrarnos tratando otra vez de expandir a A , sin haber consumido ningún símbolo de la entrada.

4.4.2 PRIMERO y SIGUIENTE

La construcción de los analizadores sintácticos descendentes y ascendentes es auxiliada por dos funciones, PRIMERO y SIGUIENTE, asociadas con la gramática G . Durante el análisis sintáctico descendente, PRIMERO y SIGUIENTE nos permiten elegir la producción que vamos a aplicar, con base en el siguiente símbolo de entrada. Durante la recuperación de errores en modo de pánico, los conjuntos de tokens que produce SIGUIENTE pueden usarse como tokens de sincronización.

Definimos a $\text{PRIMERO}(\alpha)$, en donde α es cualquier cadena de símbolos gramaticales, como el conjunto de terminales que empiezan las cadenas derivadas a partir de α . Si $\alpha \xRightarrow{*} \epsilon$, entonces ϵ también se encuentra en $\text{PRIMERO}(\alpha)$. Por ejemplo, en la figura 4.15, $A \xRightarrow{*}_{lm} c\gamma$, por lo que c está en $\text{PRIMERO}(A)$.

Para una vista previa de cómo usar PRIMERO durante el análisis sintáctico predictivo, considere dos producciones $A, A \rightarrow \alpha \mid \beta$, en donde $\text{PRIMERO}(\alpha)$ y $\text{PRIMERO}(\beta)$ son conjuntos

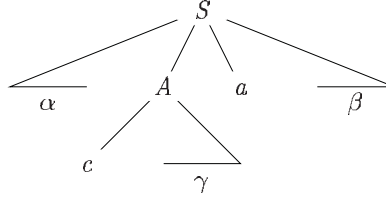


Figura 4.15: El terminal c está en $\text{PRIMERO}(A)$ y a está en $\text{SIGUIENTE}(A)$

separados. Entonces, podemos elegir una de estas producciones A si analizamos el siguiente símbolo de entrada a , ya que a puede estar a lo más en $\text{PRIMERO}(\alpha)$ o en $\text{PRIMERO}(\beta)$, pero no en ambos. Por ejemplo, si a está en $\text{PRIMERO}(\beta)$, elegimos la producción $A \rightarrow \beta$. Exploraremos esta idea en la sección 4.4.3, cuando definamos las gramáticas LL(1).

Definimos a $\text{SIGUIENTE}(A)$, para el no terminal A , como el conjunto de terminales a que pueden aparecer de inmediato a la derecha de A en cierta forma de frase; es decir, el conjunto de terminales A de tal forma que exista una derivación de la forma $S \xRightarrow{*} \alpha A a \beta$, para algunas α y β , como en la figura 4.15. Observe que pudieron haber aparecido símbolos entre A y a , en algún momento durante la derivación, pero si es así, derivaron a ϵ y desaparecieron. Además, si A puede ser el símbolo por la derecha en cierta forma de frase, entonces $\$$ está en $\text{SIGUIENTE}(A)$; recuerde que $\$$ es un símbolo “delimitador” especial, el cual se supone que no es un símbolo de ninguna gramática.

Para calcular $\text{PRIMERO}(X)$ para todos los símbolos gramaticales X , aplicamos las siguientes reglas hasta que no pueden agregarse más terminales o ϵ a ningún conjunto PRIMERO .

1. Si X es un terminal, entonces $\text{PRIMERO}(X) = \{X\}$.
2. Si X es un no terminal y $X \rightarrow Y_1 Y_2 \dots Y_k$ es una producción para cierta $k \geq 1$, entonces se coloca a en $\text{PRIMERO}(X)$ si para cierta i , a está en $\text{PRIMERO}(Y_i)$, y ϵ está en todas las funciones $\text{PRIMERO}(Y_1), \dots, \text{PRIMERO}(Y_{i-1})$; es decir, $Y_1 \dots Y_{i-1} \xRightarrow{*} \epsilon$. Si ϵ está en $\text{PRIMERO}(Y_j)$ para todas las $j = 1, 2, \dots, k$, entonces se agrega ϵ a $\text{PRIMERO}(X)$. Por ejemplo, todo lo que hay en $\text{PRIMERO}(Y_1)$ se encuentra sin duda en $\text{PRIMERO}(X)$. Si Y_1 no deriva a ϵ , entonces no agregamos nada más a $\text{PRIMERO}(X)$, pero si $Y_1 \xRightarrow{*} \epsilon$, entonces agregamos $\text{PRIMERO}(Y_2)$, y así sucesivamente.
3. Si $X \rightarrow \epsilon$ es una producción, entonces se agrega ϵ a $\text{PRIMERO}(X)$.

Ahora, podemos calcular PRIMERO para cualquier cadena $X_1 X_2 \dots X_n$ de la siguiente manera. Se agregan a $\text{PRIMERO}(X_1 X_2 \dots X_n)$ todos los símbolos que no sean ϵ de $\text{PRIMERO}(X_1)$. También se agregan los símbolos que no sean ϵ de $\text{PRIMERO}(X_2)$, si ϵ está en $\text{PRIMERO}(X_1)$; los símbolos que no sean ϵ de $\text{PRIMERO}(X_3)$, si ϵ está en $\text{PRIMERO}(X_1)$ y $\text{PRIMERO}(X_2)$; y así sucesivamente. Por último, se agrega ϵ a $\text{PRIMERO}(X_1 X_2 \dots X_n)$ si, para todas las i , ϵ se encuentra en $\text{PRIMERO}(X_i)$.

Para calcular $\text{SIGUIENTE}(A)$ para todas las no terminales A , se aplican las siguientes reglas hasta que no pueda agregarse nada a cualquier conjunto SIGUIENTE .

1. Colocar $\$$ en $\text{SIGUIENTE}(S)$, en donde S es el símbolo inicial y $\$$ es el delimitador derecho de la entrada.

2. Si hay una producción $A \rightarrow \alpha B \beta$, entonces todo lo que hay en $\text{PRIMERO}(\beta)$ excepto ϵ está en $\text{SIGUIENTE}(B)$.
3. Si hay una producción $A \rightarrow \alpha B$, o una producción $A \rightarrow \alpha B \beta$, en donde $\text{PRIMERO}(\beta)$ contiene a ϵ , entonces todo lo que hay en $\text{SIGUIENTE}(A)$ está en $\text{SIGUIENTE}(B)$.

Ejemplo 4.30: Considere de nuevo la gramática no recursiva por la izquierda (4.28). Entonces:

1. $\text{PRIMERO}(F) = \text{PRIMERO}(T) = \text{PRIMERO}(E) = \{ (, \text{id} \}$. Para ver por qué, observe que las dos producciones para F tienen producciones que empiezan con estos dos símbolos terminales, id y el paréntesis izquierdo. T sólo tiene una producción y empieza con F . Como F no deriva a ϵ , $\text{PRIMERO}(T)$ debe ser igual que $\text{PRIMERO}(F)$. El mismo argumento se cumple con $\text{PRIMERO}(E)$.
2. $\text{PRIMERO}(E') = \{ +, \epsilon \}$. La razón es que una de las dos producciones para E' tiene un cuerpo que empieza con el terminal $+$, y la otra es ϵ . Cada vez que un no terminal deriva a ϵ , colocamos a ϵ en PRIMERO para ese no terminal.
3. $\text{PRIMERO}(T') = \{ *, \epsilon \}$. El razonamiento es análogo al de $\text{PRIMERO}(E')$.
4. $\text{SIGUIENTE}(E) = \text{SIGUIENTE}(E') = \{), \$ \}$. Como E es el símbolo inicial, $\text{SIGUIENTE}(E)$ debe contener $\$$. El cuerpo de la producción (E) explica por qué el paréntesis derecho está en $\text{SIGUIENTE}(E)$. Para E' , observe que esta no terminal sólo aparece en los extremos de los cuerpos de las producciones E . Por ende, $\text{SIGUIENTE}(E')$ debe ser el mismo que $\text{SIGUIENTE}(E)$.
5. $\text{SIGUIENTE}(T) = \text{SIGUIENTE}(T') = \{ +,), \$ \}$. Observe que T aparece en las producciones sólo seguido por E' . Por lo tanto, todo lo que esté en $\text{PRIMERO}(E')$, excepto ϵ , debe estar en $\text{SIGUIENTE}(T)$; eso explica el símbolo $+$. No obstante, como $\text{PRIMERO}(E')$ contiene a ϵ (es decir, $E' \xrightarrow{*} \epsilon$), y E' es la cadena completa que va después de T en los cuerpos de las producciones E , todo lo que hay en $\text{SIGUIENTE}(E)$ también debe estar en $\text{SIGUIENTE}(T)$. Eso explica los símbolos $\$$ y el paréntesis derecho. En cuanto a T' , como aparece sólo en los extremos de las producciones T , debe ser que $\text{SIGUIENTE}(T') = \text{SIGUIENTE}(T)$.
6. $\text{SIGUIENTE}(F) = \{ +, *,), \$ \}$. El razonamiento es análogo al de T en el punto (5).

□

4.4.3 Gramáticas LL(1)

Los analizadores sintácticos predictivos, es decir, los analizadores sintácticos de descenso recursivo que no necesitan rastreo hacia atrás, pueden construirse para una clase de gramáticas llamadas LL(1). La primera “L” en LL(1) es para explorar la entrada de izquierda a derecha (por left en inglés), la segunda “L” para producir una derivación por la izquierda, y el “1” para usar un símbolo de entrada de anticipación en cada paso, para tomar las decisiones de acción del análisis sintáctico.

Diagramas de transición para analizadores sintácticos predictivos

Los diagramas de transición son útiles para visualizar los analizadores sintácticos predictivos. Por ejemplo, los diagramas de transición para las no terminales E y E' de la gramática (4.28) aparecen en la figura 4.16(a). Para construir el diagrama de transición a partir de una gramática, primero hay que eliminar la recursividad por la izquierda y después factorizar la gramática por la izquierda. Entonces, para cada no terminal A ,

1. Se crea un estado inicial y un estado final (retorno).
2. Para cada producción $A \rightarrow X_1 X_2 \dots X_k$, se crea una ruta desde el estado inicial hasta el estado final, con los flancos etiquetados como $X_1, X_2, \dots, X_k$. Si $A \rightarrow \epsilon$, la ruta es una línea que se etiqueta como ϵ .

Los diagramas de transición para los analizadores sintácticos predictivos difieren de los diagramas para los analizadores léxicos. Los analizadores sintácticos tienen un diagrama para cada no terminal. Las etiquetas de las líneas pueden ser tokens o no terminales. Una transición sobre un token (terminal) significa que tomamos esa transición si ese token es el siguiente símbolo de entrada. Una transición sobre un no terminal A es una llamada al procedimiento para A .

Con una gramática LL(1), la ambigüedad acerca de si se debe tomar o no una línea ϵ puede resolverse si hacemos que las transiciones ϵ sean la opción predeterminada.

Los diagramas de transición pueden simplificarse, siempre y cuando se preserve la secuencia de símbolos gramaticales a lo largo de las rutas. También podemos sustituir el diagrama para un no terminal A , en vez de una línea etiquetada como A . Los diagramas en las figuras 4.16(a) y (b) son equivalentes: si trazamos rutas de E a un estado de aceptación y lo sustituimos por E' , entonces, en ambos conjuntos de diagramas, los símbolos gramaticales a lo largo de las rutas forman cadenas del tipo $T + T + \dots + T$. El diagrama en (b) puede obtenerse a partir de (a) mediante transformaciones semejantes a las de la sección 2.5.4, en donde utilizamos la eliminación de recursividad de la parte final y la sustitución de los cuerpos de los procedimientos para optimizar el procedimiento en un no terminal.

La clase de gramáticas LL(1) es lo bastante robusta como para cubrir la mayoría de las construcciones de programación, aunque hay que tener cuidado al escribir una gramática adecuada para el lenguaje fuente. Por ejemplo, ninguna gramática recursiva por la izquierda o ambigua puede ser LL(1).

Una gramática G es LL(1) si, y sólo si cada vez que $A \rightarrow \alpha \mid \beta$ son dos producciones distintas de G , se aplican las siguientes condiciones:

1. Para el no terminal a , tanto α como β derivan cadenas que empiecen con a .
2. A lo más, sólo α o β puede derivar la cadena vacía.
3. Si $\beta \xRightarrow{*} \epsilon$, entonces α no deriva a ninguna cadena que empiece con una terminal en SIGUIENTE(A). De igual forma, si $\alpha \xRightarrow{*} \epsilon$, entonces β no deriva a ninguna cadena que empiece con una terminal en SIGUIENTE(A).

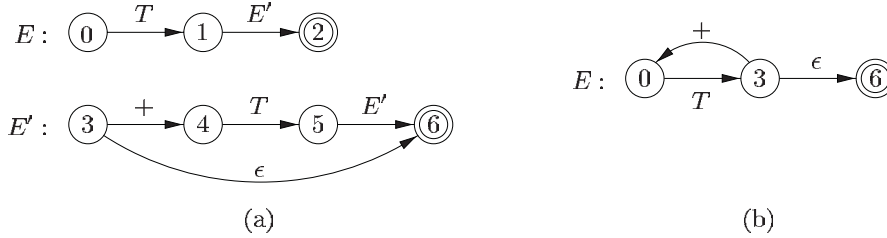


Figura 4.16: Diagramas de transición para los no terminales E y E' de la gramática 4.28

Las primeras dos condiciones son equivalentes para la instrucción que establece que $\text{PRIMERO}(\alpha)$ y $\text{PRIMERO}(\beta)$ son conjuntos separados. La tercera condición equivale a decir que si ϵ está en $\text{PRIMERO}(\beta)$, entonces $\text{PRIMERO}(\alpha)$ y $\text{SIGUIENTE}(A)$ son conjuntos separados, y de igual forma si ϵ está en $\text{PRIMERO}(\alpha)$.

Pueden construirse analizadores sintácticos predictivos para las gramáticas LL(1), ya que puede seleccionarse la producción apropiada a aplicar para una no terminal con sólo analizar el símbolo de entrada actual. Los constructores del flujo de control, con sus palabras clave distintivas, por lo general, cumplen con las restricciones de LL(1). Por ejemplo, si tenemos las siguientes producciones:

$$\begin{array}{lcl}
 \text{instr} & \rightarrow & \mathbf{if} \ (\text{expr}) \ \text{instr} \ \mathbf{else} \ \text{instr} \\
 & | & \mathbf{while} \ (\text{expr}) \ \text{instr} \\
 & | & \{ \text{lista_instr} \}
 \end{array}$$

entonces las palabras clave **if**, **while** y el símbolo $\{$ nos indican qué alternativa es la única que quizá podría tener éxito, si vamos a buscar una instrucción.

El siguiente algoritmo recolecta la información de los conjuntos PRIMERO y SIGUIENTE en una tabla de análisis predictivo $M[A, a]$, un arreglo bidimensional, en donde A es un no terminal y a es un terminal o el símbolo $\$$, el marcador de fin de la entrada. El algoritmo se basa en la siguiente idea: se elige la producción $A \rightarrow \alpha$ si el siguiente símbolo de entrada a se encuentra en $\text{PRIMERO}(\alpha)$. La única complicación ocurre cuando $\alpha = \epsilon$, o en forma más general, $\alpha \xrightarrow{*} \epsilon$. En este caso, debemos elegir de nuevo $A \rightarrow \alpha$ si el símbolo de entrada actual se encuentra en $\text{SIGUIENTE}(A)$, o si hemos llegado al $\$$ en la entrada y $\$$ se encuentra en $\text{SIGUIENTE}(A)$.

Algoritmo 4.31: Construcción de una tabla de análisis sintáctico predictivo.

ENTRADA: La gramática G .

SALIDA: La tabla de análisis sintáctico M .

MÉTODO: Para cada producción $A \rightarrow \alpha$ de la gramática, hacer lo siguiente:

1. Para cada terminal a en $\text{PRIMERO}(A)$, agregar $A \rightarrow \alpha$ a $M[A, a]$.
2. Si ϵ está en $\text{PRIMERO}(\alpha)$, entonces para cada terminal b en $\text{SIGUIENTE}(A)$, se agrega $A \rightarrow \alpha$ a $M[A, b]$. Si ϵ está en $\text{PRIMERO}(\alpha)$ y $\$$ se encuentra en $\text{SIGUIENTE}(A)$, se agrega $A \rightarrow \alpha$ a $M[A, \$]$ también.

Si después de realizar lo anterior, no hay producción en $M[A, a]$, entonces se establece $M[A, a]$ a **error** (que, por lo general, representamos mediante una entrada vacía en la tabla). $\square$

Ejemplo 4.32: Para la gramática de expresiones (4.28), el Algoritmo 4.31 produce la tabla de análisis sintáctico en la figura 4.17. Los espacios en blanco son entradas de error; los espacios que no están en blanco indican una producción con la cual se expande un no terminal.

No TERMINAL	SÍMBOLO DE ENTRADA					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Figura 4.17: Tabla de análisis sintáctico M para el ejemplo 4.32

Considere la producción $E \rightarrow TE'$. Como

$$\text{PRIMERO}(TE') = \text{PRIMERO}(T) = \{ (, \text{id} \}$$

esta producción se agrega a $M[E, (]$ y $M[E, \text{id}]$. La producción $E' \rightarrow +TE'$ se agrega a $M[E', +]$, ya que $\text{PRIMERO}(+TE') = \{ + \}$. Como $\text{SIGUIENTE}(E') = \{), \$ \}$, la producción $E' \rightarrow \epsilon$ se agrega a $M[E',)]$ y a $M[E', \$]$. $\square$

El algoritmo 4.31 puede aplicarse a cualquier gramática G para producir una tabla de análisis sintáctico M . Para cada gramática LL(1), cada entrada en la tabla de análisis sintáctico identifica en forma única a una producción, o indica un error. Sin embargo, para algunas gramáticas, M puede tener algunas entradas que tengan múltiples definiciones. Por ejemplo, si G es recursiva por la izquierda o ambigua, entonces M tendrá por lo menos una entrada con múltiples definiciones. Aunque la eliminación de la recursividad por la izquierda y la factorización por la izquierda son fáciles de realizar, hay algunas gramáticas para las cuales ningún tipo de alteración producirá una gramática LL(1).

El lenguaje en el siguiente ejemplo no tiene una gramática LL(1).

Ejemplo 4.33: La siguiente gramática, que abstrae el problema del else colgante, se repite aquí del ejemplo 4.22:

$$\begin{aligned} S &\rightarrow iEtSS' \mid a \\ S' &\rightarrow eS \mid \epsilon \\ E &\rightarrow b \end{aligned}$$

La tabla de análisis sintáctico para esta gramática aparece en la figura 4.18. La entrada para $M[S', e]$ contiene tanto a $S' \rightarrow eS$ como a $S' \rightarrow \epsilon$.

La gramática es ambigua y la ambigüedad se manifiesta mediante una elección de qué producción usar cuando se ve una e (**else**). Podemos resolver esta ambigüedad eligiendo $S' \rightarrow eS$.

No TERMINAL	SÍMBOLO DE ENTRADA					
	a	b	ϵ	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow \epsilon S$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Figura 4.18: Tabla de análisis sintáctico M para el ejemplo 4.33

Esta elección corresponde a la asociación de un **else** con el **then** anterior más cercano. Observe que la elección $S' \rightarrow \epsilon$ evitaría que ϵ se metiera en la pila o se eliminara de la entrada, y eso definitivamente está mal. $\square$

4.4.4 Análisis sintáctico predictivo no recursivo

Podemos construir un analizador sintáctico predictivo no recursivo mediante el mantenimiento explícito de una pila, en vez de hacerlo mediante llamadas recursivas implícitas. El analizador sintáctico imita una derivación por la izquierda. Si w es la entrada que se ha relacionado hasta ahora, entonces la pila contiene una secuencia de símbolos gramaticales α de tal forma que:

$$S \xRightarrow[im]{*} w\alpha$$

El analizador sintáctico controlado por una tabla, que se muestra en la figura 4.19, tiene un búfer de entrada, una pila que contiene una secuencia de símbolos gramaticales, una tabla de análisis sintáctico construida por el Algoritmo 4.31, y un flujo de salida. El búfer de entrada contiene la cadena que se va a analizar, seguida por el marcador final $\$$. Reutilizamos el símbolo $\$$ para marcar la parte inferior de la pila, que al principio contiene el símbolo inicial de la gramática encima de $\$$.

El analizador sintáctico se controla mediante un programa que considera a X , el símbolo en la parte superior de la pila, y a a , el símbolo de entrada actual. Si X es un no terminal, el analizador sintáctico elige una producción X mediante una consulta a la entrada $M[X, a]$ de la tabla de análisis sintáctico M (aquí podría ejecutarse código adicional; por ejemplo, el código para construir un nodo en un árbol de análisis sintáctico). En cualquier otro caso, verifica si hay una coincidencia entre el terminal X y el símbolo de entrada actual a .

El comportamiento del analizador sintáctico puede describirse en términos de sus *configuraciones*, que proporcionan el contenido de la pila y el resto de la entrada. El siguiente algoritmo describe la forma en que se manipulan las configuraciones.

Algoritmo 4.34: Análisis sintáctico predictivo, controlado por una tabla.

ENTRADA: Una cadena w y una tabla de análisis sintáctico M para la gramática G .

SALIDA: Si w está en $L(G)$, una derivación por la izquierda de w ; en caso contrario, una indicación de error.

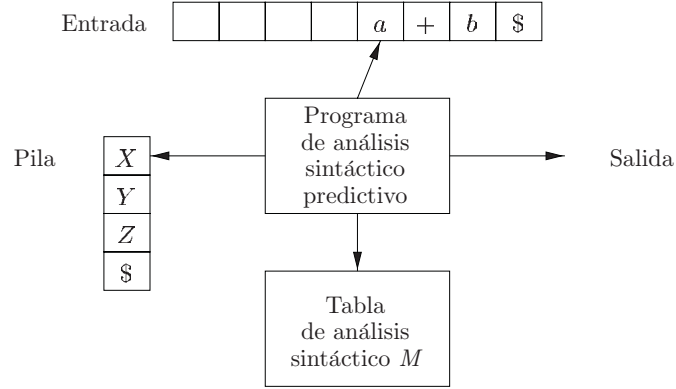


Figura 4.19: Modelo de un analizador sintáctico predictivo, controlado por una tabla

MÉTODO: Al principio, el analizador sintáctico se encuentra en una configuración con $w\$$ en el búfer de entrada, y el símbolo inicial S de G en la parte superior de la pila, por encima de $\$$. El programa en la figura 4.20 utiliza la tabla de análisis sintáctico predictivo M para producir un análisis sintáctico predictivo para la entrada. $\square$

```

establecer  $ip$  para que apunte al primer símbolo de  $w$ ;
establecer  $X$  con el símbolo de la parte superior de la pila;
while (  $X \neq \$$  ) { /* la pila no está vacía */
    if (  $X$  es  $a$  ) sacar de la pila y avanzar  $ip$ ;
    else if (  $X$  es un terminal )  $error()$ ;
    else if (  $M[X, a]$  es una entrada de error )  $error()$ ;
    else if (  $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$  ) {
        enviar de salida la producción  $X \rightarrow Y_1 Y_2 \dots Y_k$ ;
        sacar de la pila;
        meter  $Y_k, Y_{k-1}, \dots, Y_1$  en la pila, con  $Y_1$  en la parte superior;
    }
    establecer  $X$  con el símbolo de la parte superior de la pila;
}

```

Figura 4.20: Algoritmo de análisis sintáctico predictivo

Ejemplo 4.35: Considere la gramática (4.28); se muestra la tabla de análisis sintáctico en la figura 4.17. Con la entrada $\mathbf{id} + \mathbf{id} * \mathbf{id}$, el analizador predictivo sin recursividad del Algoritmo 4.34 realiza la secuencia de movimientos en la figura 4.21. Estos movimientos corresponden a una derivación por la izquierda (vea la figura 4.12 para la derivación completa):

$$E \xRightarrow{lm} TE' \xRightarrow{lm} FT'E' \xRightarrow{lm} \mathbf{id} T'E' \xRightarrow{lm} \mathbf{id} E' \xRightarrow{lm} \mathbf{id} + TE' \xRightarrow{lm} \dots$$

COINCIDENCIA	PILA	ENTRADA	ACCIÓN
	$E\$$	$\text{id} + \text{id} * \text{id}\$$	
	$TE'\$$	$\text{id} + \text{id} * \text{id}\$$	emitir $E \rightarrow TE'$
	$FT'E'\$$	$\text{id} + \text{id} * \text{id}\$$	emitir $T \rightarrow FT'$
	$\text{id } T'E'\$$	$\text{id} + \text{id} * \text{id}\$$	emitir $F \rightarrow \text{id}$
id	$T'E'\$$	$+ \text{id} * \text{id}\$$	relacionar id
id	$E'\$$	$+ \text{id} * \text{id}\$$	emitir $T' \rightarrow \epsilon$
id	$+ TE'\$$	$+ \text{id} * \text{id}\$$	emitir $E' \rightarrow + TE'$
$\text{id} +$	$TE'\$$	$\text{id} * \text{id}\$$	relacionar $+$
$\text{id} +$	$FT'E'\$$	$\text{id} * \text{id}\$$	emitir $T \rightarrow FT'$
$\text{id} +$	$\text{id } T'E'\$$	$\text{id} * \text{id}\$$	emitir $F \rightarrow \text{id}$
$\text{id} + \text{id}$	$T'E'\$$	$* \text{id}\$$	relacionar id
$\text{id} + \text{id}$	$* FT'E'\$$	$* \text{id}\$$	emitir $T' \rightarrow * FT'$
$\text{id} + \text{id} *$	$FT'E'\$$	$\text{id}\$$	relacionar $*$
$\text{id} + \text{id} *$	$\text{id } T'E'\$$	$\text{id}\$$	emitir $F \rightarrow \text{id}$
$\text{id} + \text{id} * \text{id}$	$T'E'\$$	$\$$	relacionar id
$\text{id} + \text{id} * \text{id}$	$E'\$$	$\$$	emitir $T' \rightarrow \epsilon$
$\text{id} + \text{id} * \text{id}$	$\$$	$\$$	emitir $E' \rightarrow \epsilon$

Figura 4.21: Movimientos que realiza un analizador sintáctico predictivo con la entrada $\text{id} + \text{id} * \text{id}$

Observe que las formas de frases en esta derivación corresponden a la entrada que ya se ha relacionado (en la columna COINCIDENCIA), seguida del contenido de la pila. La entrada relacionada se muestra sólo para resaltar la correspondencia. Por la misma razón, la parte superior de la pila está a la izquierda; cuando consideremos el análisis sintáctico ascendente, será más natural mostrar la parte superior de la pila a la derecha. El apuntador de la entrada apunta al símbolo por la izquierda de la cadena en la columna ENTRADA. $\square$

4.4.5 Recuperación de errores en el análisis sintáctico predictivo

Esta discusión sobre la recuperación de errores se refiere a la pila de un analizador sintáctico predictivo controlado por una tabla, ya que hace explícitas los terminales y no terminales que el analizador sintáctico espera relacionar con el resto de la entrada; las técnicas también pueden usarse con el análisis sintáctico de descenso recursivo.

Durante el análisis sintáctico predictivo, un error se detecta cuando el terminal en la parte superior de la pila no coincide con el siguiente símbolo de entrada, o cuando el no terminal A se encuentra en la parte superior de la pila, a es el siguiente símbolo de entrada y $M[A, a]$ es **error** (es decir, la entrada en la tabla de análisis sintáctico está vacía).

Modo de pánico

La recuperación de errores en modo de pánico se basa en la idea de omitir símbolos en la entrada hasta que aparezca un token en un conjunto seleccionado de tokens de sincronización. Su

efectividad depende de la elección del conjunto de sincronización. Los conjuntos deben elegirse de forma que el analizador sintáctico se recupere con rapidez de los errores que tengan una buena probabilidad de ocurrir en la práctica. Algunas heurísticas son:

1. Como punto inicial, colocar todos los símbolos que están en $\text{SIGUIENTE}(A)$ en el conjunto de sincronización para el no terminal A . Si omitimos tokens hasta que se vea un elemento de $\text{SEGUIMIENTO}(A)$ y sacamos a A de la pila, es probable que el análisis sintáctico pueda continuar.
2. No basta con usar $\text{SIGUIENTE}(A)$ como el conjunto de sincronización para A . Por ejemplo, si los signos de punto y coma terminan las instrucciones, como en C, entonces las palabras reservadas que empiezan las instrucciones no pueden aparecer en el conjunto SIGUIENTE del no terminal que representa a las expresiones. Un punto y coma faltante después de una asignación puede, por lo tanto, ocasionar que se omita la palabra reservada que empieza la siguiente instrucción. A menudo hay una estructura jerárquica en las construcciones en un lenguaje; por ejemplo, las expresiones aparecen dentro de las instrucciones, las cuales aparecen dentro de bloques, y así sucesivamente. Al conjunto de sincronización de una construcción de bajo nivel podemos agregar los símbolos que empiezan las construcciones de un nivel más alto. Por ejemplo, podríamos agregar palabras clave que empiezan las instrucciones a los conjuntos de sincronización para los no terminales que generan las expresiones.
3. Si agregamos los símbolos en $\text{PRIMERO}(A)$ al conjunto de sincronización para el no terminal A , entonces puede ser posible continuar con el análisis sintáctico de acuerdo con A , si en la entrada aparece un símbolo que se encuentre en $\text{PRIMERO}(A)$.
4. Si un no terminal puede generar la cadena vacía, entonces la producción que deriva a ϵ puede usarse como predeterminada. Al hacer esto se puede posponer cierta detección de errores, pero no se puede provocar la omisión de un error. Este método reduce el número de terminales que hay que considerar durante la recuperación de errores.
5. Si un terminal en la parte superior de la pila no se puede relacionar, una idea simple es sacar el terminal, emitir un mensaje que diga que se insertó el terminal, y continuar con el análisis sintáctico. En efecto, este método requiere que el conjunto de sincronización de un token consista de todos los demás tokens.

Ejemplo 4.36: El uso de los símbolos en PRIMERO y SIGUIENTE como tokens de sincronización funciona razonablemente bien cuando las expresiones se analizan de acuerdo con la gramática usual (4.28). La tabla de análisis sintáctico para esta gramática de la figura 4.17 se repite en la figura 4.22, en donde “sinc” indica los tokens de sincronización obtenidos del conjunto SIGUIENTE de la no terminal en cuestión. Los conjuntos SIGUIENTE para las no terminales se obtienen del ejemplo 4.30.

La tabla en la figura 4.22 debe usarse de la siguiente forma. Si el analizador sintáctico busca la entrada $M[A, a]$ y descubre que está en blanco, entonces se omite el símbolo de entrada a . Si la entrada es “sinc”, entonces se saca el no terminal que está en la parte superior de la pila, en un intento por continuar con el análisis sintáctico. Si un token en la parte superior de la pila no coincide con el símbolo de entrada, entonces sacamos el token de la pila, como dijimos antes.

No TERMINAL	SÍMBOLO DE ENTRADA					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	sinc	sinc
E'		$E \rightarrow +TE'$			$E \rightarrow \epsilon$	$E \rightarrow \epsilon$
T	$T \rightarrow FT'$	sinc		$T \rightarrow FT'$	sinc	sinc
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \mathbf{id}$	sinc	sinc	$F \rightarrow (E)$	sinc	sinc

Figura 4.22: Tokens de sincronización agregados a la tabla de análisis sintáctico de la figura 4.17

Con la entrada errónea **)id * + id**, el analizador sintáctico y el mecanismo de recuperación de errores de la figura 4.22 se comporta como en la figura 4.23. $\square$

PILA	ENTRADA	COMENTARIO
$E \$$	) id * + id \$	error, omitir)
$E \$$	id * + id \$	id está en PRIMERO(E)
$TE' \$$	id * + id \$	
$FT'E' \$$	id * + id \$	
id $T'E' \$$	id * + id \$	
$T'E' \$$	* + id \$	
* $FT'E' \$$	* + id \$	
$FT'E' \$$	+ id \$	error, $M[F, +] = \text{sinc}$
$T'E' \$$	+ id \$	Se sacó F
$E' \$$	+ id \$	
+ $TE' \$$	+ id \$	
$TE' \$$	id \$	
$FT'E' \$$	id \$	
id $T'E' \$$	id \$	
$T'E' \$$	\$	
$E' \$$	\$	
\$	\$	

Figura 4.23: Movimientos de análisis sintáctico y recuperación de errores realizados por un analizador sintáctico predictivo

La discusión anterior sobre la recuperación en modo de pánico no señala el punto importante relacionado con los mensajes de error. El diseñador del compilador debe proporcionar mensajes de error informativos que no sólo describan el error, sino que también llamen la atención hacia el lugar en donde se descubrió el error.

Recuperación a nivel de frase

La recuperación de errores a nivel de frase se implementa llenando las entradas en blanco en la tabla de análisis sintáctico predictivo con apuntes a rutinas de error. Estas rutinas pueden modificar, insertar o eliminar símbolos en la entrada y emitir mensajes de error apropiados. También pueden sacar de la pila. La alteración de los símbolos de la pila o el proceso de meter nuevos símbolos a la pila es cuestionable por dos razones. En primer lugar, los pasos que realiza el analizador sintáctico podrían entonces no corresponder a la derivación de ninguna palabra en el lenguaje. En segundo lugar, debemos asegurarnos de que no haya posibilidad de un ciclo infinito. Verificar que cualquier acción de recuperación ocasione en un momento dado que se consuma un símbolo de entrada (o que se reduzca la pila si se ha llegado al fin de la entrada) es una buena forma de protegerse contra tales ciclos.

4.4.6 Ejercicios para la sección 4.4

Ejercicio 4.4.1: Para cada una de las siguientes gramáticas, idee analizadores sintácticos predictivos y muestre las tablas de análisis sintáctico. Puede factorizar por la izquierda o eliminar la recursividad por la izquierda de sus gramáticas primero.

- a) La gramática del ejercicio 4.2.2(a).
- b) La gramática del ejercicio 4.2.2(b).
- c) La gramática del ejercicio 4.2.2(c).
- d) La gramática del ejercicio 4.2.2(d).
- e) La gramática del ejercicio 4.2.2(e).
- f) La gramática del ejercicio 4.2.2(g).

!! Ejercicio 4.4.2: ¿Es posible, mediante la modificación de la gramática en cualquier forma, construir un analizador sintáctico predictivo para el lenguaje del ejercicio 4.2.1 (expresiones postfijo con el operando a)?

Ejercicio 4.4.3: Calcule PRIMERO y SIGUIENTE para la gramática del ejercicio 4.2.1.

Ejercicio 4.4.4: Calcule PRIMERO y SIGUIENTE para cada una de las gramáticas del ejercicio 4.2.2.

Ejercicio 4.4.5: La gramática $S \rightarrow a S a \mid a a$ genera todas las cadenas de longitud uniforme de as . Podemos idear un analizador sintáctico de descenso recursivo con rastreo hacia atrás para esta gramática. Si elegimos expandir mediante la producción $S \rightarrow a a$ primero, entonces sólo debemos reconocer la cadena aa . Por ende, cualquier analizador sintáctico de descenso recursivo razonable probará $S \rightarrow a S a$ primero.

- a) Muestre que este analizador sintáctico de descenso recursivo reconoce las entradas aa , $aaaa$ y $aaaaaaaa$, pero no $aaaaaa$.

!! b) ¿Qué lenguaje reconoce este analizador sintáctico de descenso recursivo?

Los siguientes ejercicios son pasos útiles en la construcción de una gramática en la “Forma Normal de Chomsky” a partir de gramáticas arbitrarias, como se define en el ejercicio 4.4.8.

! Ejercicio 4.4.6: Una gramática es *libre de ϵ* si ningún cuerpo de las producciones es ϵ (a lo cual se le llama *producción ϵ*).

- Proporcione un algoritmo para convertir cualquier gramática en una gramática libre de ϵ que genere el mismo lenguaje (con la posible excepción de la cadena vacía; ninguna gramática libre de ϵ puede generar a ϵ).
- Aplique su algoritmo a la gramática $S \rightarrow aSbS \mid bSaS \mid \epsilon$. *Sugerencia:* Primero busque todas las no terminales que sean *anulables*, lo cual significa que generan a ϵ , tal vez mediante una derivación extensa.

! Ejercicio 4.4.7: Una *producción simple* es una producción cuyo cuerpo es una sola no terminal; por ejemplo, una producción de la forma $A \rightarrow A$.

- Proporcione un algoritmo para convertir cualquier gramática en una gramática libre de ϵ , sin producciones simples, que genere el mismo lenguaje (con la posible excepción de la cadena vacía) *Sugerencia:* Primero elimine las producciones ϵ y después averigüe para qué pares de no terminales A y B se cumple que $A \xRightarrow{*} B$ mediante una secuencia de producciones simples.
- Aplique su algoritmo a la gramática (4.1) en la sección 4.1.2.
- Muestre que, como consecuencia de la parte (a), podemos convertir una gramática en una gramática equivalente que no tenga *ciclos* (derivaciones de uno o más pasos, en los que $A \xRightarrow{*} A$ para cierta no terminal A).

!! Ejercicio 4.4.8: Se dice que una gramática está en *Forma Normal de Chomsky* (FNC) si toda producción es de la forma $A \rightarrow BC$ o de la forma $A \rightarrow a$, en donde A , B y C son no terminales, y a es un terminal. Muestre cómo convertir cualquier gramática en una gramática FNC para el mismo lenguaje (con la posible excepción de la cadena vacía; ninguna gramática FNC puede generar a ϵ).

! Ejercicio 4.4.9: Todo lenguaje que tiene una gramática libre de contexto puede reconocerse en un tiempo máximo de $O(n^3)$ para las cadenas de longitud n . Una manera simple de hacerlo, conocida como el algoritmo de *Cocke-Younger-Kasami* (o CYK), se basa en la programación dinámica. Es decir, dada una cadena $a_1a_2 \cdots a_n$, construimos una tabla T de n por n de tal forma que T_{ij} sea el conjunto de no terminales que generen la subcadena $a_ia_{i+1} \cdots a_j$. Si la gramática subyacente está en FNC (vea el ejercicio 4.4.8), entonces una entrada en la tabla puede llenarse en un tiempo $O(n)$, siempre y cuando llenemos las entradas en el orden apropiado: el menor valor de $j - i$ primero. Escriba un algoritmo que llene en forma correcta las entradas de la tabla, y muestre que su algoritmo requiere un tiempo $O(n^3)$. Después de llenar la tabla, ¿cómo podemos determinar si $a_1a_2 \cdots a_n$ está en el lenguaje?

! **Ejercicio 4.4.10:** Muestre cómo, después de llenar la tabla como en el ejercicio 4.4.9, podemos recuperar en un tiempo $O(n)$ un árbol de análisis sintáctico para $a_1 a_2 \cdots a_n$. *Sugerencia:* Modifique la tabla de manera que registre, para cada no terminal A en cada entrada de la tabla T_{ij} , cierto par de no terminales en otras entradas en la tabla que justifiquen la acción de colocar a A en T_{ij} .

! **Ejercicio 4.4.11:** Modifique su algoritmo del ejercicio 4.4.9 de manera que busque, para cualquier cadena, el menor número de errores de inserción, eliminación y mutación (cada error de un solo carácter) necesarios para convertir la cadena en una cadena del lenguaje de la gramática subyacente.

$instr$	$\rightarrow$	if e then $instr$ $colaInstr$
		while e do $instr$
		begin $lista$ end
		s
$colaInstr$	$\rightarrow$	else $instr$
		ϵ
$lista$	$\rightarrow$	$instr$ $colaLista$
$colaLista$	$\rightarrow$	$;$ $lista$
	$\rightarrow$	ϵ

Figura 4.24: Una gramática para ciertos tipos de instrucciones

! **Ejercicio 4.4.12:** En la figura 4.24 hay una gramática para ciertas instrucciones. Podemos considerar que e y s son terminales que representan expresiones condicionales y “otras instrucciones”, respectivamente. Si resolvemos el conflicto en relación con la expansión de la instrucción “else” opcional (la no terminal $colaInstr$) al preferir consumir un **else** de la entrada cada vez que veamos uno, podemos construir un analizador sintáctico predictivo para esta gramática. Usando la idea de los símbolos de sincronización descritos en la sección 4.4.5:

- a) Construya una tabla de análisis sintáctico predictivo con corrección de errores para la gramática.
- b) Muestre el comportamiento de su analizador sintáctico con las siguientes entradas:

(i) **if** e **then** s ; **if** e **then** s **end**
(ii) **while** e **do** **begin** s ; **if** e **then** s ; **end**

4.5 Análisis sintáctico ascendente

Un análisis sintáctico ascendente corresponde a la construcción de un árbol de análisis sintáctico para una cadena de entrada que empieza en las hojas (la parte inferior) y avanza hacia la raíz (la parte superior). Es conveniente describir el análisis sintáctico como el proceso de construcción de árboles de análisis sintáctico, aunque de hecho un front-end de usuario podría realizar una traducción directamente, sin necesidad de construir un árbol explícito. La secuencia

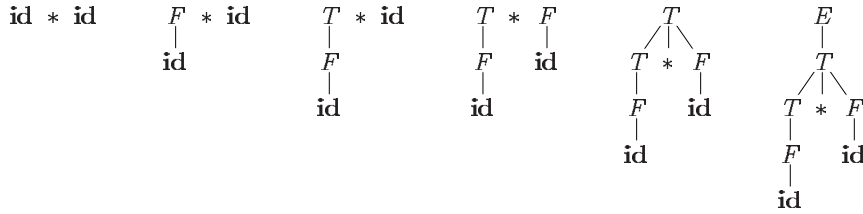


Figura 4.25: Un análisis sintáctico ascendente para $\text{id} * \text{id}$

de imágenes de árboles en la figura 4.25 ilustra un análisis sintáctico ascendente del flujo de tokens $\text{id} * \text{id}$, con respecto a la gramática de expresiones (4.1).

Esta sección presenta un estilo general de análisis sintáctico ascendente, conocido como análisis sintáctico de desplazamiento-reducción. En las secciones 4.6 y 4.7 hablaremos sobre las gramáticas LR, la clase más extensa de gramáticas para las cuales pueden construirse los analizadores sintácticos de desplazamiento-reducción. Aunque es demasiado trabajo construir un analizador sintáctico LR en forma manual, las herramientas conocidas como generadores automáticos de analizadores sintácticos facilitan la construcción de analizadores sintácticos LR eficientes a partir de gramáticas adecuadas. Los conceptos en esta sección son útiles para escribir gramáticas adecuadas que nos permitan hacer un uso efectivo de un generador de analizadores sintácticos LR. En la sección 4.7 aparecen los algoritmos para implementar los generadores de analizadores sintácticos.

4.5.1 Reducciones

Podemos considerar el análisis sintáctico ascendente como el proceso de “reducir” una cadena w al símbolo inicial de la gramática. En cada paso de *reducción*, se sustituye una subcadena específica que coincide con el cuerpo de una producción por el no terminal que se encuentra en el encabezado de esa producción.

Las decisiones clave durante el análisis sintáctico ascendente son acerca de cuándo reducir y qué producción aplicar, a medida que procede el análisis sintáctico.

Ejemplo 4.37: Las imágenes en la figura 4.25 ilustran una secuencia de reducciones; la gramática es la gramática de expresiones (4.1). Hablaremos sobre las reducciones en términos de la siguiente secuencia de cadenas:

$$\text{id} * \text{id}, F * \text{id}, T * \text{id}, T * F, T, E$$

Las cadenas en esta secuencia se forman a partir de las raíces de todos los subárboles de las imágenes. La secuencia empieza con la cadena de entrada $\text{id} * \text{id}$. La primera reducción produce $F * \text{id}$ al reducir el id por la izquierda a F , usando la producción $F \rightarrow \text{id}$. La segunda reducción produce $T * \text{id}$ al reducir F a T .

Ahora tenemos una elección entre reducir la cadena T , que es el cuerpo de $E \rightarrow T$, y la cadena que consiste en el segundo id , que es el cuerpo de $F \rightarrow \text{id}$. En vez de reducir T a E , el segundo id se reduce a T , con lo cual se produce la cadena $T * F$. Después, esta cadena se reduce a T . El análisis sintáctico termina con la reducción de T al símbolo inicial E . $\square$

Por definición, una reducción es el inverso de un paso en una derivación (recuerde que en una derivación, un no terminal en una forma de frase se sustituye por el cuerpo de una de sus producciones). Por lo tanto, el objetivo del análisis sintáctico ascendente es construir una derivación en forma inversa. La siguiente derivación corresponde al análisis sintáctico en la figura 4.25:

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

Esta derivación es de hecho una derivación por la derecha.

4.5.2 Poda de mangos

Durante una exploración de izquierda a derecha de la entrada, el análisis sintáctico ascendente construye una derivación por la derecha en forma inversa. De manera informal, un “mango” es una subcadena que coincide con el cuerpo de una producción, y cuya reducción representa un paso a lo largo del inverso de una derivación por la derecha.

Por ejemplo, si agregamos subíndices a los tokens $\mathbf{id}$ para mejorar la legibilidad, los mangos durante el análisis sintáctico de $\mathbf{id}_1 * \mathbf{id}_2$, de acuerdo con la gramática de expresiones (4.1), son como en la figura 4.26. Aunque T es el cuerpo de la producción $E \rightarrow T$, el símbolo T no es un mango en la forma de frase $T * \mathbf{id}_2$. Si T se sustituyera por E , obtendríamos la cadena $E * \mathbf{id}_2$, lo cual no puede derivarse del símbolo inicial E . Por ende, la subcadena por la izquierda que coincide con el cuerpo de alguna producción no necesita ser un mango.

FORMA DE FRASE DERECHA	MANGO	REDUCCIÓN DE LA PRODUCCIÓN
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \rightarrow \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \rightarrow F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \rightarrow \mathbf{id}$
$T * F$	$T * F$	$E \rightarrow T * F$

Figura 4.26: Mangos durante un análisis sintáctico de $\mathbf{id}_1 * \mathbf{id}_2$

De manera formal, si $S \xRightarrow{*} \alpha A w \Rightarrow \alpha \beta w$, como en la figura 4.27, entonces la producción $A \rightarrow \beta$ en la posición que sigue después de α es un *mango* de $\alpha \beta w$. De manera alternativa, un mango de la forma de frase derecha γ es una producción $A \rightarrow \beta$ y una posición de γ en donde puede encontrarse la cadena β , de tal forma que al sustituir β en esa posición por A se produzca la forma de frase derecha anterior en una derivación por la derecha de γ .

Observe que la cadena w a la derecha del mango debe contener sólo símbolos terminales. Por conveniencia, nos referimos al cuerpo β en vez de $A \rightarrow \beta$ como un mango. Observe que decimos “un mango” en vez de “el mango”, ya que la gramática podría ser ambigua, con más de una derivación por la derecha de $\alpha \beta w$. Si una gramática no tiene ambigüedad, entonces cada forma de frase derecha de la gramática tiene sólo un mango.

Puede obtenerse una derivación por la derecha en forma inversa mediante la “poda de mangos”. Es decir, empezamos con una cadena de terminales w a las que se les va a realizar

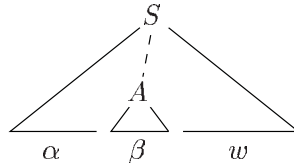


Figura 4.27: Un mango $A \rightarrow \beta$ en el árbol de análisis sintáctico para $\alpha\beta w$

el análisis sintáctico. Si w es un enunciado de la gramática a la mano, entonces dejamos que $w = \gamma_n$, en donde γ_n es la n -ésima forma de frase derecha de alguna derivación por la derecha, que todavía se desconoce:

$$S = \gamma_0 \xRightarrow{rm} \gamma_1 \xRightarrow{rm} \gamma_2 \xRightarrow{rm} \cdots \xRightarrow{rm} \gamma_{n-1} \xRightarrow{rm} \gamma_n = w$$

Para reconstruir esta derivación en orden inverso, localizamos el mango β_n en γ_n y sustituimos β_n por el encabezado de la producción $A_n \rightarrow \beta_n$ para obtener la forma de frase derecha γ_{n-1} anterior. Tenga en cuenta que todavía no sabemos cómo se van a encontrar los mangos, pero en breve veremos métodos para hacerlo.

Después repetimos este proceso. Es decir, localizamos el mango β_{n-1} en γ_{n-1} y reducimos este mango para obtener la forma de frase derecha γ_{n-2} . Si al continuar este proceso producimos una forma de frase derecha que consista sólo en el símbolo inicial S , entonces nos detenemos y anunciamos que el análisis sintáctico se completó con éxito. El inverso de la secuencia de producciones utilizadas en las reducciones es una derivación por la derecha para la cadena de entrada.

4.5.3 Análisis sintáctico de desplazamiento-reducción

El análisis sintáctico de desplazamiento-reducción es una forma de análisis sintáctico ascendente, en la cual una pila contiene símbolos gramaticales y un búfer de entrada contiene el resto de la cadena que se va a analizar. Como veremos, el mango siempre aparece en la parte superior de la pila, justo antes de identificarla como el mango.

Utilizamos el $\$$ para marcar la parte inferior de la pila y también el extremo derecho de la entrada. Por convención, al hablar sobre el análisis sintáctico ascendente, mostramos la parte superior de la pila a la derecha, en vez de a la izquierda como hicimos para el análisis sintáctico descendente. Al principio la pila está vacía, y la cadena w está en la entrada, como se muestra a continuación:

PILA	ENTRADA
\$	w \$

Durante una exploración de izquierda a derecha de la cadena de entrada, el analizador sintáctico desplaza cero o más símbolos de entrada y los mete en la pila, hasta que esté listo para reducir una cadena β de símbolos gramaticales en la parte superior de la pila. Después reduce β al encabezado de la producción apropiada. El analizador sintáctico repite este ciclo hasta que haya detectado un error, o hasta que la pila contenga el símbolo inicial y la entrada esté vacía:

PILA	ENTRADA
\$ S	\$

Al entrar a esta configuración, el analizador sintáctico se detiene y anuncia que el análisis sintáctico se completó con éxito. La figura 4.28 avanza por pasos a través de las acciones que podría realizar un analizador sintáctico de desplazamiento-reducción al analizar la cadena de entrada $\mathbf{id}_1 * \mathbf{id}_2$, de acuerdo con la gramática de expresiones (4.1).

PILA	ENTRADA	ACCIÓN
\$	$\mathbf{id}_1 * \mathbf{id}_2$ \$	desplazar
\$ $\mathbf{id}_1$	* $\mathbf{id}_2$ \$	reducir $F \rightarrow \mathbf{id}$
\$ F	* $\mathbf{id}_2$ \$	reducir $T \rightarrow F$
\$ T	* $\mathbf{id}_2$ \$	desplazar
\$ T *	$\mathbf{id}_2$ \$	desplazar
\$ $T * \mathbf{id}_2$	\$	reducir $F \rightarrow \mathbf{id}$
\$ $T * F$	\$	reducir $T \rightarrow T * F$
\$ T	\$	reducir $E \rightarrow T$
\$ E	\$	aceptar

Figura 4.28: Configuraciones de un analizador sintáctico de desplazamiento-reducción, con una entrada $\mathbf{id}_1 * \mathbf{id}_2$

Aunque las operaciones primarias son desplazar y reducir, en realidad hay cuatro acciones posibles que puede realizar un analizador sintáctico de desplazamiento-reducción: (1) desplazar, (2) reducir, (3) aceptar y (4) error.

1. *Desplazar*. Desplazar el siguiente símbolo de entrada y lo coloca en la parte superior de la pila.
2. *Reducir*. El extremo derecho de la cadena que se va a reducir debe estar en la parte superior de la pila. Localizar el extremo izquierdo de la cadena dentro de la pila y decidir con qué terminal se va a sustituir la cadena.
3. *Aceptar*. Anunciar que el análisis sintáctico se completó con éxito.
4. *Error*. Descubrir un error de sintaxis y llamar a una rutina de recuperación de errores.

El uso de una pila en el análisis sintáctico de desplazamiento-reducción se justifica debido a un hecho importante: el mango siempre aparecerá en algún momento dado en la parte superior de la pila, nunca en el interior. Este hecho puede demostrarse si consideramos las posibles formas de dos pasos sucesivos en cualquier derivación por la izquierda. La figura 4.29 ilustra los dos posibles casos. En el caso (1), A se sustituye por βBy , y después el no terminal B por la derecha en el cuerpo βBy se sustituye por γ . En el caso (2), A se expande primero otra vez, pero ahora el cuerpo es una cadena y que consiste sólo en terminales. El siguiente no terminal B por la derecha se encontrará en alguna parte a la derecha de y .

En otras palabras:

$$\begin{aligned}
 (1) \quad S &\xRightarrow[rm]{*} \alpha Az \Rightarrow \alpha \beta Byz \Rightarrow \alpha \beta \gamma yz \\
 (2) \quad S &\xRightarrow[rm]{*} \alpha Bx Az \Rightarrow \alpha Bxyz \Rightarrow \alpha \gamma xyz
 \end{aligned}$$

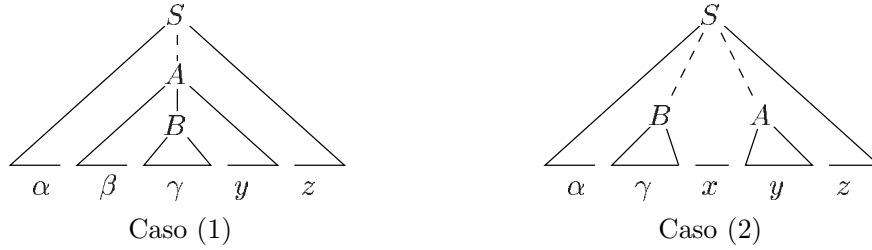


Figura 4.29: Casos para dos pasos sucesivos de una derivación por la derecha

Considere el caso (1) a la inversa, en donde un analizador sintáctico de desplazamiento-reducción acaba de llegar a la siguiente configuración:

PILA	ENTRADA
$\$ \alpha \beta \gamma$	$y z \$$

El analizador sintáctico reduce el mango γ a B para llegar a la siguiente configuración:

$\$ \alpha \beta B$	$y z \$$
---------------------	----------

Ahora el analizador puede desplazar la cadena y y colocarla en la pila, mediante una secuencia de cero o más movimientos de desplazamiento para llegar a la configuración

$\$ \alpha \beta B y$	$z \$$
-----------------------	--------

con el mango $\beta B y$ en la parte superior de la pila, y se reduce a A .

Ahora considere el caso (2). En la configuración

$\$ \alpha \gamma$	$x y z \$$
--------------------	------------

el mango γ está en la parte superior de la pila. Después de reducir el mango γ a B , el analizador sintáctico puede reducir la cadena xy para meter el siguiente mango y en la parte superior de la pila, listo para reducirse a A :

$\$ \alpha B x y$	$z \$$
-------------------	--------

En ambos casos, después de realizar una reducción, el analizador sintáctico tuvo que desplazar cero o más símbolos para meter el siguiente mango en la pila. Nunca tuvo que buscar el mango dentro de la pila.

4.5.4 Conflictos durante el análisis sintáctico de desplazamiento-reducción

Existen gramáticas libres de contexto para las cuales no se puede utilizar el análisis sintáctico de desplazamiento-reducción. Cada analizador sintáctico de desplazamiento-reducción para una gramática de este tipo puede llegar a una configuración en la cual el analizador sintáctico, conociendo el contenido completo de la pila y el siguiente símbolo de entrada, no puede decidir

si va a desplazar o a reducir (un *conflicto de desplazamiento/reducción*), o no puede decidir qué reducciones realizar (un *conflicto de reducción/reducción*). Ahora veremos algunos ejemplos de construcciones sintácticas que ocasionan tales gramáticas. Técnicamente, estas gramáticas no están en la clase $LR(k)$ de gramáticas definidas en la sección 4.7; nos referimos a ellas como gramáticas no LR. La k en $LR(k)$ se refiere al número de símbolos de preanálisis en la entrada. Las gramáticas que se utilizan en la compilación, por lo general, entran en la clase $LR(1)$, con un símbolo de anticipación a lo más.

Ejemplo 4.38: Una gramática ambigua nunca podrá ser LR. Por ejemplo, considere la gramática del *else* colgante (4.14) de la sección 4.3:

$$\begin{array}{lcl} instr & \rightarrow & \text{if } expr \text{ then } instr \\ & & | \text{ if } expr \text{ then } instr \text{ else } instr \\ & & | \text{ otra} \end{array}$$

Si tenemos un analizador sintáctico de desplazamiento-reducción con la siguiente configuración:

PILA	ENTRADA
... if <i>expr</i> then <i>instr</i>	else ... \$

no podemos saber si **if** *expr* **then** *instr* es el mango, sin importar lo que aparezca debajo de él en la pila. Aquí tenemos un conflicto de desplazamiento/reducción. Dependiendo de lo que siga después del **else** en la entrada, podría ser correcto reducir **if** *expr* **then** *instr* a *instr*, o podría ser correcto desplazar el **else** y después buscar otro *instr* para completar la expresión alternativa **if** *expr* **then** *instr* **else** *instr*.

Observe que el análisis sintáctico de desplazamiento-reducción puede adaptarse para analizar ciertas gramáticas ambiguas, como la gramática if-then-else anterior. Si resolvemos el conflicto de desplazamiento/reducción en el **else** a favor del desplazamiento, el analizador sintáctico se comportará como esperamos, asociando cada **else** con el **then** anterior sin coincidencia. En la sección 4.8 hablaremos sobre los analizadores sintácticos para dichas gramáticas ambiguas. $\square$

Otra configuración común para los conflictos ocurre cuando sabemos que tenemos un mango, pero el contenido de la pila y el siguiente símbolo de entrada no son suficientes para determinar qué producción debe usarse en una reducción. El siguiente ejemplo ilustra esta situación.

Ejemplo 4.39: Suponga que tenemos un analizador léxico que devuelve el nombre de token **id** para todos los nombres, sin importar su tipo. Suponga también que nuestro lenguaje invoca a los procedimientos proporcionando sus nombres, con los parámetros rodeados entre paréntesis, y que los arreglos se referencian mediante la misma sintaxis. Como la traducción de los índices en las referencias a arreglos y los parámetros en las llamadas a procedimientos son distintos, queremos usar distintas producciones para generar listas de parámetros e índices actuales. Por lo tanto, nuestra gramática podría tener producciones como las de la figura 4.30 (entre otras).

Una instrucción que empieza con **p(i, j)** aparecería como el flujo de tokens **id(id, id)** para el analizador sintáctico. Después de desplazar los primeros tres tokens en la pila, un analizador sintáctico de desplazamiento-reducción tendría la siguiente configuración:

(1)	<i>instr</i>	$\rightarrow$	id (<i>lista_parametros</i>)
(2)	<i>instr</i>	$\rightarrow$	<i>expr</i> := <i>expr</i>
(3)	<i>lista_parametros</i>	$\rightarrow$	<i>lista_parametros</i> , <i>parámetro</i>
(4)	<i>lista_parametros</i>	$\rightarrow$	<i>parámetro</i>
(5)	<i>parametro</i>	$\rightarrow$	id
(6)	<i>expr</i>	$\rightarrow$	id (<i>lista_expr</i>)
(7)	<i>expr</i>	$\rightarrow$	id
(8)	<i>lista_expr</i>	$\rightarrow$	<i>lista_expr</i> , <i>expr</i>
(9)	<i>lista_expr</i>	$\rightarrow$	<i>expr</i>

Figura 4.30: Producciones que implican llamadas a procedimientos y referencias a arreglos

PILA	ENTRADA
... id (id	, id) ...

Es evidente que el **id** en la parte superior de la pila debe reducirse, pero ¿mediante qué producción? La elección correcta es la producción (5) si **p** es un procedimiento, pero si **p** es un arreglo, entonces es la producción (7). La pila no indica qué información debemos usar en la en la tabla de símbolos que se obtiene a partir de la declaración de **p**.

Una solución es cambiar el token **id** en la producción (1) a **procid** y usar un analizador léxico más sofisticado, que devuelva el nombre de token **procid** cuando reconozca un lexema que sea el nombre de un procedimiento. Para ello se requeriría que el analizador léxico consultara la tabla de símbolos, antes de devolver un token.

Si realizamos esta modificación, entonces al procesar $p(i,j)$ el analizador se encontraría en la siguiente configuración:

PILA	ENTRADA
... procid (id	, id) ...

o en la configuración anterior. En el caso anterior, elegimos la reducción mediante la producción (5); en el último caso mediante la producción (7). Observe cómo el tercer símbolo de la parte superior de la pila determina la reducción que se va a realizar, aun cuando no está involucrado en la reducción. El análisis sintáctico de desplazamiento-reducción puede utilizar información de más adentro en la pila, para guiar el análisis sintáctico. $\square$

4.5.5 Ejercicios para la sección 4.5

Ejercicio 4.5.1: Para la gramática $S \rightarrow 0S1 \mid 01$ del ejercicio 4.2.2(a), indique el mango en cada una de las siguientes formas de frases derechas:

- a) 000111.
- b) 00S11.

Ejercicio 4.5.2: Repita el ejercicio 4.5.1 para la gramática $S \rightarrow SS+ \mid SS* \mid a$ del ejercicio 4.2.1 y las siguientes formas de frases derechas:

- a) $SSS + a * +$.
- b) $SS + a * a +$.
- c) $aaa * a + +$.

Ejercicio 4.5.3: Proporcione los análisis sintácticos ascendentes para las siguientes cadenas de entrada y gramáticas:

- a) La entrada 000111, de acuerdo a la gramática del ejercicio 4.5.1.
- b) La entrada $aaa * a++$, de acuerdo a la gramática del ejercicio 4.5.2.

4.6 Introducción al análisis sintáctico LR: SLR (LR simple)

El tipo más frecuente de analizador sintáctico ascendentes en la actualidad se basa en un concepto conocido como análisis sintáctico LR(k); la “L” indica la exploración de izquierda a derecha de la entrada, la “R” indica la construcción de una derivación por la derecha a la inversa, y la k para el número de símbolos de entrada de preanálisis que se utilizan al hacer decisiones del análisis sintáctico. Los casos $k = 0$ o $k = 1$ son de interés práctico, por lo que aquí sólo consideraremos los analizadores sintácticos LR con $k \leq 1$. Cuando se omite (k), se asume que k es 1.

Esta sección presenta los conceptos básicos del análisis sintáctico LR y el método más sencillo para construir analizadores sintácticos de desplazamiento-reducción, llamados “LR Simple” (o SLR). Es útil tener cierta familiaridad con los conceptos básicos, incluso si el analizador sintáctico LR se construye mediante un generador de analizadores sintácticos automático. Empezaremos con “elementos” y “estados del analizador sintáctico”; la salida de diagnóstico de un generador de analizadores sintácticos LR, por lo general, incluye estados del analizador sintáctico, los cuales pueden usarse para aislar las fuentes de conflictos en el análisis sintáctico.

La sección 4.7 introduce dos métodos más complejos (LR canónico y LALR) que se utilizan en la mayoría de los analizadores sintácticos LR.

4.6.1 ¿Por qué analizadores sintácticos LR?

Los analizadores sintácticos LR son controlados por tablas, en forma muy parecida a los analizadores sintácticos LL no recursivos de la sección 4.4.4. Se dice que una gramática para la cual podemos construir una tabla de análisis sintáctico, usando uno de los métodos en esta sección y en la siguiente, es una *gramática LR*. De manera intuitiva, para que una gramática sea LR, basta con que un analizador sintáctico de desplazamiento-reducción de izquierda a derecha pueda reconocer mangos de las formas de frases derechas, cuando éstas aparecen en la parte superior de la pila.

El análisis sintáctico LR es atractivo por una variedad de razones:

- Pueden construirse analizadores sintácticos LR para reconocer prácticamente todas las construcciones de lenguajes de programación para las cuales puedan escribirse gramáticas libres de contexto. Existen gramáticas libres de contexto que no son LR, pero por lo general se pueden evitar para las construcciones comunes de los lenguajes de programación.

- El método de análisis sintáctico LR es el método de análisis sintáctico de desplazamiento-reducción sin rastreo hacia atrás más general que se conoce a la fecha, y aún así puede implementarse con la misma eficiencia que otros métodos más primitivos de desplazamiento-reducción (vea las notas bibliográficas).
- Un analizador sintáctico LR puede detectar un error sintáctico tan pronto como sea posible en una exploración de izquierda a derecha de la entrada.
- La clase de gramáticas que pueden analizarse mediante los métodos LR es un superconjunto propio de la clase de gramáticas que pueden analizarse con métodos predictivos o LL. Para que una gramática sea $LR(k)$, debemos ser capaces de reconocer la ocurrencia del lado derecho de una producción en una forma de frase derecha, con k símbolos de entrada de preanálisis. Este requerimiento es mucho menos estricto que para las gramáticas $LL(k)$, en donde debemos ser capaces de reconocer el uso de una producción, viendo sólo los primeros símbolos k de lo que deriva su lado derecho. Por ende, no debe sorprender que las gramáticas LR puedan describir más lenguajes que las gramáticas LL.

La principal desventaja del método LR es que es demasiado trabajo construir un analizador sintáctico LR en forma manual para una gramática común de un lenguaje de programación. Se necesita una herramienta especializada: un generador de analizadores sintácticos LR. Por fortuna, hay varios generadores disponibles, y en la sección 4.9 hablaremos sobre uno de los que se utilizan con más frecuencia: *Yacc*. Dicho generador recibe una gramática libre de contexto y produce de manera automática un analizador para esa gramática. Si la gramática contiene ambigüedades u otras construcciones que sean difíciles de analizar en una exploración de izquierda a derecha de la entrada, entonces el generador de analizadores sintácticos localiza estas construcciones y proporciona mensajes de diagnóstico detallados.

4.6.2 Los elementos y el autómata $LR(0)$

¿Cómo sabe un analizador sintáctico de desplazamiento-reducción cuándo desplazar y cuándo reducir? Por ejemplo, con el contenido T de la pila y el siguiente símbolo de entrada $*$ en la figura 4.28, ¿cómo sabe el analizador sintáctico que la T en la parte superior de la pila no es un mango, por lo cual la acción apropiada es desplazar y no reducir T a E ?

Un analizador sintáctico LR realiza las decisiones de desplazamiento-reducción mediante el mantenimiento de estados, para llevar el registro de la ubicación que tenemos en un análisis sintáctico. Los estados representan conjuntos de “elementos”. Un *elemento* $LR(0)$ (*elemento*, para abreviar) de una gramática G es una producción de G con un punto en cierta posición del cuerpo. Por ende, la producción $A \rightarrow XYZ$ produce los siguientes cuatro elementos:

$$\begin{aligned} A &\rightarrow \cdot XYZ \\ A &\rightarrow X \cdot YZ \\ A &\rightarrow XY \cdot Z \\ A &\rightarrow XYZ \cdot \end{aligned}$$

La producción $A \rightarrow \epsilon$ genera sólo un elemento, $A \rightarrow \cdot$.

De manera intuitiva, un elemento indica qué parte de una producción hemos visto en un punto dado del proceso de análisis sintáctico. Por ejemplo, el elemento $A \rightarrow \cdot XYZ$ indica

Representación de conjuntos de elementos

Un generador de análisis sintáctico que produce un analizador descendente tal vez requiera representar elementos y conjuntos de elementos en una forma conveniente. Un elemento puede representarse mediante un par de enteros, el primero de los cuales es el número de una de las producciones de la gramática subyacente, y el segundo de los cuales es la posición del punto. Los conjuntos de elementos pueden representarse mediante una lista de estos pares. No obstante, como veremos pronto, los conjuntos necesarios de elementos a menudo incluyen elementos de “cierre”, en donde el punto se encuentra al principio del cuerpo. Estos siempre pueden reconstruirse a partir de los otros elementos en el conjunto, por lo que no tenemos que incluirlos en la lista.

que esperamos ver una cadena que pueda derivarse de XYZ a continuación en la entrada. El elemento $A \rightarrow X \cdot YZ$ indica que acabamos de ver en la entrada una cadena que puede derivarse de X , y que esperamos ver a continuación una cadena que pueda derivarse de $Y Z$. El elemento $A \rightarrow XYZ \cdot$ indica que hemos visto el cuerpo XYZ y que puede ser hora de reducir XYZ a A .

Una colección de conjuntos de elementos $LR(0)$, conocida como la colección $LR(0)$ *canónica*, proporciona la base para construir un autómata finito determinista, el cual se utiliza para realizar decisiones en el análisis sintáctico. A dicho autómata se le conoce como *autómata $LR(0)$* .³ En especial, cada estado del autómata $LR(0)$ representa un conjunto de elementos en la colección $LR(0)$ canónica. El autómata para la gramática de expresiones (4.1), que se muestra en la figura 4.31, servirá como ejemplo abierto para hablar sobre la colección $LR(0)$ canónica para una gramática.

Para construir la colección $LR(0)$ canónica de una gramática, definimos una gramática aumentada y dos funciones, $CERRADURA$ e ir_A . Si G es una gramática con el símbolo inicial S , entonces G' , la *gramática aumentada* para G , es G con un nuevo símbolo inicial S' y la producción $S' \rightarrow S$. El propósito de esta nueva producción inicial es indicar al analizador sintáctico cuándo debe dejar de analizar para anunciar la aceptación de la entrada. Es decir, la aceptación ocurre sólo cuando el analizador sintáctico está a punto de reducir mediante $S' \rightarrow S$.

Cerradura de conjuntos de elementos

Si I es un conjunto de elementos para una gramática G , entonces $CERRADURA(I)$ es el conjunto de elementos que se construyen a partir de I mediante las siguientes dos reglas:

1. Al principio, agregar cada elemento en I a $CERRADURA(I)$.
2. Si $A \rightarrow \alpha \cdot B \beta$ está en $CERRADURA(I)$ y $B \rightarrow \gamma$ es una producción, entonces agregar el elemento $B \rightarrow \gamma$ a $CERRADURA(I)$, si no se encuentra ya ahí. Aplicar esta regla hasta que no puedan agregarse más elementos nuevos a $CERRADURA(I)$.

³Técnicamente, el autómata deja de ser determinista de acuerdo a la definición de la sección 3.6.4, ya que no tenemos un estado muerto, que corresponde al conjunto vacío de elementos. Como resultado, hay ciertos pares estado-entrada para los cuales no existe un siguiente estado.

Para ver cómo se calcula la CERRADURA, $E' \rightarrow \cdot E$ se coloca en $\text{CERRADURA}(I)$ mediante la regla (1). Como hay una E justo a la derecha de un punto, agregamos las producciones E con puntos en los extremos izquierdos: $E \rightarrow \cdot E + T$ y $E \rightarrow \cdot T$. Ahora hay una T justo a la derecha de un punto en este último elemento, por lo que agregamos $T \rightarrow \cdot T * F$ y $T \rightarrow \cdot F$. A continuación, la F a la derecha de un punto nos obliga a agregar $F \rightarrow \cdot (E)$ y $F \rightarrow \cdot \text{id}$, pero no necesita agregarse ningún otro elemento. $\square$

La cerradura puede calcularse como en la figura 4.32. Una manera conveniente de implementar la función *cerradura* es mantener un arreglo booleano llamado *agregado*, indexado mediante los no terminales de G , de tal forma que $\text{agregado}[B]$ se establezca a **true** si, y sólo si agregamos el elemento $B \rightarrow \cdot \gamma$ para cada producción B de la forma $B \rightarrow \gamma$.

```

ConjuntoDeElementos CERRADURA( $I$ ) {
     $J = I$ ;
    repeat
        for ( cada elemento  $A \rightarrow \alpha \cdot B \beta$  en  $J$  )
            for ( cada producción  $B \rightarrow \gamma$  de  $G$  )
                if (  $B \rightarrow \cdot \gamma$  no está en  $J$  )
                    agregar  $B \rightarrow \cdot \gamma$  a  $J$ ;
    until no se agreguen más elementos a  $J$  en una ronda;
    return  $J$ ;
}

```

Figura 4.32: Cálculo de CERRADURA

Observe que si se agrega una producción B al cierre de I con el punto en el extremo izquierdo, entonces se agregarán todas las producciones B de manera similar al cierre. Por ende, no es necesario en algunas circunstancias listar los elementos $B \rightarrow \gamma$ que se agregan a I mediante CERRADURA. Basta con una lista de los no terminales B cuyas producciones se agregaron. Dividimos todos los conjuntos de elementos de interés en dos clases:

1. *Elementos del corazón*: el elemento inicial, $S' \rightarrow \cdot S$, y todos los elementos cuyos puntos no estén en el extremo izquierdo.
2. *Elementos que no son del corazón*: todos los elementos con sus puntos en el extremo izquierdo, excepto $S' \rightarrow \cdot S$.

Además, cada conjunto de elementos de interés se forma tomando la cerradura de un conjunto de elementos del corazón; desde luego que los elementos que se agregan en la cerradura nunca podrán ser elementos del corazón. Por lo tanto, podemos representar los conjuntos de elementos en los que realmente estamos interesados con muy poco almacenamiento si descartamos todos los elementos que no sean del corazón, sabiendo que podrían regenerarse mediante el proceso de cerradura. En la figura 4.31, los elementos que no son del corazón se encuentran en la parte sombreada del cuadro para un estado.

La función ir_A

La segunda función útil es $\text{ir_A}(I, X)$, en donde I es un conjunto de elementos y X es un símbolo gramatical. $\text{ir_A}(I, X)$ se define como la cerradura del conjunto de todos los elementos $[A \rightarrow \alpha X \beta]$, de tal forma que $[A \rightarrow \alpha X \beta]$ se encuentre en I . De manera intuitiva, la función ir_A se utiliza para definir las transiciones en el autómata LR(0) para una gramática. Los estados del autómata corresponden a los conjuntos de elementos, y $\text{ir_A}(I, X)$ especifica la transición que proviene del estado para I , con la entrada X .

Ejemplo 4.41: Si I es el conjunto de dos elementos $\{[E' \rightarrow E\cdot], [E \rightarrow E\cdot + T]\}$, entonces $\text{ir_A}(I, +)$ contiene los siguientes elementos:

$$\begin{array}{ll} E & \rightarrow E + \cdot T \\ T & \rightarrow \cdot T * F \\ T & \rightarrow \cdot F \\ F & \rightarrow \cdot (E) \\ F & \rightarrow \cdot \text{id} \end{array}$$

Para calcular $\text{ir_A}(I, +)$, examinamos I en busca de elementos con $+$ justo a la derecha del punto. $E' \rightarrow E\cdot$ no es uno de estos elementos, pero $E \rightarrow E\cdot + T$ sí lo es. Desplazamos el punto sobre el $+$ para obtener $E \rightarrow E + \cdot T$ y después tomamos la cerradura de este conjunto singleton. $\square$

Ahora estamos listos para que el algoritmo construya a C , la colección canónica de conjuntos de elementos LR(0) para una gramática aumentada G' ; el algoritmo se muestra en la figura 4.33.

```

void elementos( $G'$ ) {
     $C = \text{CERRADURA}(\{[S' \rightarrow \cdot S]\})$ ;
    repeat
        for ( cada conjunto de elementos  $I$  en  $C$  )
            for ( cada símbolo gramatical  $X$  )
                if (  $\text{ir\_A}(I, X)$  no está vacío y no está en  $C$  )
                    agregar  $\text{ir\_A}(I, X)$  a  $C$ ;
    until no se agreguen nuevos conjuntos de elementos a  $C$  en una iteración;
}

```

Figura 4.33: Cálculo de la colección canónica de conjuntos de elementos LR(0)

Ejemplo 4.42: La colección canónica de conjuntos de elementos LR(0) para la gramática (4.1) y la función ir_A se muestran en la figura 4.31. ir_A se codifica mediante las transiciones en la figura. $\square$

Uso del autómata LR(0)

La idea central del análisis sintáctico “LR simple”, o SLR, es la construcción del autómata LR(0) a partir de la gramática. Los estados de este autómata son los conjuntos de elementos de la colección LR(0) canónica, y las traducciones las proporciona la función ir_A . El autómata LR(0) para la gramática de expresiones (4.1) apareció antes en la figura 4.31.

El estado inicial del autómata LR(0) es CERRADURA($\{[S' \rightarrow \cdot S]\}$), en donde S' es el símbolo inicial de la gramática aumentada. Todos los estados son de aceptaciones. Decimos que el “estado j ” se refiere al estado que corresponde al conjunto de elementos I_j .

¿Cómo puede ayudar el autómata LR(0) con las decisiones de desplazar-reducir? Estas decisiones pueden realizarse de la siguiente manera. Suponga que la cadena γ de símbolos gramaticales lleva el autómata LR(0) del estado inicial 0 a cierto estado j . Después, se realiza un desplazamiento sobre el siguiente símbolo de entrada a si el estado j tiene una transición en a . En cualquier otro caso, elegimos reducir; los elementos en el estado j nos indicarán qué producción usar.

El algoritmo de análisis sintáctico LR que presentaremos en la sección 4.6.3 utiliza su pila para llevar el registro de los estados, así como de los símbolos gramaticales; de hecho, el símbolo gramatical puede recuperarse del estado, por lo que la pila contiene los estados. El siguiente ejemplo proporciona una vista previa acerca de cómo pueden utilizarse un autómata LR(0) y una pila de estados para realizar decisiones de desplazamiento-reducción en el análisis sintáctico.

Ejemplo 4.43: La figura 4.34 ilustra las acciones de un analizador sintáctico de desplazamiento-reducción con la entrada **id * id**, usando el autómata LR(0) de la figura 4.31. Utilizamos una pila para guardar los estados; por claridad, los símbolos gramaticales que corresponden a los estados en la pila aparecen en la columna SÍMBOLOS. En la línea (1), la pila contiene el estado inicial 0 del autómata; el símbolo correspondiente es el marcador \$ de la parte inferior de la pila.

LÍNEA	PILA	SÍMBOLOS	ENTRADA	ACCIÓN
(1)	0	\$	id * id \$	desplazar 5
(2)	0 5	\$ id	* id \$	reducir $F \rightarrow \mathbf{id}$
(3)	0 3	\$ F	* id \$	reducir $T \rightarrow F$
(4)	0 2	\$ T	* id \$	desplazar 7
(5)	0 2 7	\$ $T *$	id \$	desplazar 5
(6)	0 2 7 5	\$ $T * \mathbf{id}$	\$	reducir $F \rightarrow \mathbf{id}$
(7)	0 2 7 10	\$ $T * F$	\$	reducir $T \rightarrow T * F$
(8)	0 2	\$ T	\$	reducir $E \rightarrow T$
(9)	0 1	\$ E	\$	aceptar

Figura 4.34: El análisis sintáctico de **id * id**

El siguiente símbolo de entrada es **id** y el estado 0 tiene una transición en **id** al estado 5. Por lo tanto, realizamos un desplazamiento. En la línea (2), el estado 5 (símbolo **id**) se ha metido en la pila. No hay transición desde el estado 5 con la entrada *****, por lo que realizamos una reducción. Del elemento $[F \rightarrow \mathbf{id} \cdot]$ en el estado 5, la reducción es mediante la producción $F \rightarrow \mathbf{id}$.

Con los símbolos, una reducción se implementa sacando el cuerpo de la producción de la pila (en la línea (2), el cuerpo es **id**) y metiendo el encabezado de la producción (en este caso, F). Con los estados, sacamos el estado 5 para el símbolo **id**, lo cual lleva el estado 0 a la parte superior y buscamos una transición en F , el encabezado de la producción. En la figura 4.31, el estado 0 tiene una transición en F al estado 3, por lo que metemos el estado 3, con el símbolo F correspondiente; vea la línea (3).

Como otro ejemplo, considere la línea (5), con el estado 7 (símbolo $*$) en la parte superior de la pila. Este estado tiene una transición al estado 5 con la entrada **id**, por lo que metemos el estado 5 (símbolo **id**). El estado 5 no tiene transiciones, así que lo reducimos mediante $F \rightarrow \mathbf{id}$. Al sacar el estado 5 para el cuerpo **id**, el estado 7 pasa a la parte superior de la pila. Como el estado 7 tiene una transición en F al estado 10, metemos el estado 10 (símbolo F). $\square$

4.6.3 El algoritmo de análisis sintáctico LR

En la figura 4.35 se muestra un diagrama de un analizador sintáctico LR. Este diagrama consiste en una entrada, una salida, una pila, un programa controlador y una tabla de análisis sintáctico que tiene dos partes (ACCION y el ir_A). El programa controlador es igual para todos los analizadores sintácticos LR; sólo la tabla de análisis sintáctico cambia de un analizador sintáctico a otro. El programa de análisis sintáctico lee caracteres de un búfer de entrada, uno a la vez. En donde un analizador sintáctico de desplazamiento-reducción desplazaría a un símbolo, un analizador sintáctico LR desplaza a un *estado*. Cada estado sintetiza la información contenida en la pila, debajo de éste.

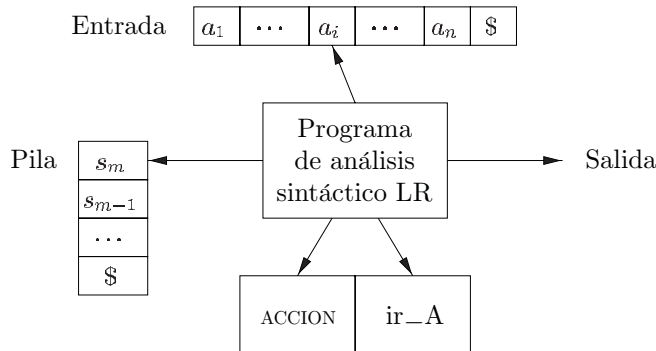


Figura 4.35: Modelo de un analizador sintáctico LR

La pila contiene una secuencia de estados, $s_0 s_1 \dots s_m$, en donde s_m , se encuentra en la parte superior. En el método SLR, la pila contiene estados del autómata $\text{LR}(0)$; los métodos LR canónico y LALR son similares. Por construcción, cada estado tiene un símbolo gramatical correspondiente. Recuerde que los estados corresponden a los conjuntos de elementos, y que hay una transición del estado i al estado j si $\text{ir_A}(I_i, X) = I_j$. Todas las transiciones al estado j deben ser para el mismo símbolo gramatical X . Por ende, cada estado, excepto el estado inicial 0, tiene un símbolo gramatical único asociado con él.⁴

⁴Lo opuesto no necesariamente es válido; es decir, más de un estado puede tener el mismo símbolo gramatical. Por

Estructura de la tabla de análisis sintáctico LR

La tabla de análisis sintáctico consiste en dos partes: una función de acción de análisis sintáctico llamada ACCION y una función ir_A.

1. La función ACCION recibe como argumentos un estado i y un terminal a (o \$, el marcador de fin de entrada). El valor de ACCION[i , a] puede tener una de cuatro formas:
 - (a) Desplazar j , en donde j es un estado. La acción realizada por el analizador sintáctico desplaza en forma efectiva la entrada a hacia la pila, pero usa el estado j para representar la a .
 - (b) Reducir $A \rightarrow \beta$. La acción del analizador reduce en forma efectiva a β en la parte superior de la pila, al encabezado A .
 - (c) Aceptar. El analizador sintáctico acepta la entrada y termina el análisis sintáctico.
 - (d) Error. El analizador sintáctico descubre un error en su entrada y realiza cierta acción correctiva. En las secciones 4.8.3 y 4.8.4 hablaremos más acerca de cómo funcionan dichas rutinas de recuperación de errores.
2. Extendemos la función ir_A, definida en los conjuntos de elementos, a los estados: si ir_A[I_i , A] = I_j , entonces ir_A también asigna un estado i y un no terminal A al estado j .

Configuraciones del analizador sintáctico LR

Para describir el comportamiento de un analizador sintáctico LR, es útil tener una notación que represente el estado completo del analizador sintáctico: su pila y el resto de la entrada. Una *configuración* de un analizador sintáctico LR es un par:

$$(s_0 s_1 \cdots s_m, a_i a_{i+1} \cdots a_n \$)$$

en donde el primer componente es el contenido de la pila (parte superior a la derecha) y el segundo componente es el resto de la entrada. Esta configuración representa la forma de frase derecha:

$$X_1 X_2 \cdots X_m a_i a_{i+1} \cdots a_n$$

básicamente en la misma forma en que lo haría un analizador sintáctico de desplazamiento-reducción; la única diferencia es que en vez de símbolos gramaticales, la pila contiene estados a partir de los cuales pueden recuperarse los símbolos gramaticales. Es decir, X_i es el símbolo gramatical representado mediante el estado s_i . Observe que s_0 , el estado inicial del analizador sintáctico, no representa a un símbolo gramatical y sirve como marcador de la parte inferior de la pila, así como también juega un papel importante en el análisis sintáctico.

ejemplo, vea los estados 1 y 8 en el autómata LR(0) de la figura 4.31, a los cuales se entra mediante las transiciones en E , o los estados 2 y 9, a los cuales se entra mediante las transiciones en T .

Comportamiento del analizador sintáctico LR

El siguiente movimiento del analizador sintáctico a partir de la configuración anterior, se determina mediante la lectura de a_i , el símbolo de entrada actual, y s_m , el estado en la parte superior de la pila, y después se consulta la entrada $\text{ACCION}[s_m, a_i]$ en la tabla de acción del análisis sintáctico. Las configuraciones resultantes después de cada uno de los cuatro tipos de movimiento son:

1. Si $\text{ACCION}[s_m, a_i] = \text{desplazar } s$, el analizador ejecuta un movimiento de desplazamiento; desplaza el siguiente estado s y lo mete en la pila, introduciendo la siguiente configuración:

$$(s_0 s_1 \cdots s_m s, a_{i+1} \cdots a_n \$)$$

El símbolo a_i no necesita guardarse en la pila, ya que puede recuperarse a partir de s , si es necesario (en la práctica, nunca lo es). Ahora, el símbolo de entrada actual es a_{i+1} .

2. Si $\text{ACCION}[s_m, a_i] = \text{reducir } A \rightarrow \beta$, entonces el analizador sintáctico ejecuta un movimiento de reducción, entrando a la siguiente configuración:

$$(s_0 s_1 \cdots s_{m-r} s, a_i a_{i+1} \cdots a_n \$)$$

en donde r es la longitud de β , y $s = \text{ir_A}[s_{m-r}, A]$. Aquí, el analizador sintáctico primero sacó los símbolos del estado r de la pila, exponiendo al estado s_{m-r} . Después el analizador sintáctico metió a s , la entrada para $\text{ir_A}[s_{m-r}, A]$ en la pila. El símbolo de entrada actual no se cambia en un movimiento de reducción. Para los analizadores sintácticos LR que vamos a construir, $X_{m-r+1} \cdots X_m$, la secuencia de los símbolos gramaticales correspondientes a los estados que se sacan de la pila, siempre coincidirá con β , el lado derecho de la producción reductora.

La salida de un analizador sintáctico LR se genera después de un movimiento de reducción, mediante la ejecución de la acción semántica asociada con la producción reductora. Por el momento, vamos a suponer que la salida consiste sólo en imprimir la producción reductora.

3. Si $\text{ACCION}[s_m, a_i] = \text{aceptar}$, se completa el análisis sintáctico.
4. Si $\text{ACCION}[s_m, a_i] = \text{error}$, el analizador ha descubierto un error y llama a una rutina de recuperación de errores.

A continuación se sintetiza el algoritmo de análisis sintáctico LR. Todos los analizadores sintácticos LR se comportan de esta manera; la única diferencia entre un analizador sintáctico LR y otro es la información en los campos ACCION e ir_A de la tabla de análisis sintáctico.

Algoritmo 4.44: Algoritmo de análisis sintáctico LR.

ENTRADA: Una cadena de entrada w y una tabla de análisis sintáctico LR con las funciones ACCION e ir_A , para una gramática G .

SALIDA: Si w está en $L(G)$, los pasos de reducción de un análisis sintáctico ascendentes para w ; en cualquier otro caso, una indicación de error.

MÉTODO: Al principio, el analizador sintáctico tiene s_0 en su pila, en donde s_0 es el estado inicial y $w\$$ está en el búfer de entrada. Entonces, el analizador ejecuta el programa en la figura 4.36. $\square$

```

hacer que  $a$  sea el primer símbolo de  $w\$$ ;
while(1) { /* repetir indefinidamente */
    hacer que  $s$  sea el estado en la parte superior de la pila;
    if ( ACCION[ $s, a$ ] = desplazar  $t$  ) {
        meter  $t$  en la pila;
        hacer que  $a$  sea el siguiente símbolo de entrada;
    } else if ( ACCION[ $s, a$ ] = reducir  $A \rightarrow \beta$  ) {
        sacar  $|\beta|$  símbolos de la pila;
        hacer que el estado  $t$  ahora esté en la parte superior de la pila;
        meter  $\text{ir\_A}[t, A]$  en la pila;
        enviar de salida la producción  $A \rightarrow \beta$ ;
    } else if ( ACCION[ $s, a$ ] = aceptar ) break; /* terminó el análisis sintáctico */
    else llamar a la rutina de recuperación de errores;
}

```

Figura 4.36: Programa de análisis sintáctico LR

Ejemplo 4.45: La figura 4.37 muestra las funciones ACCION e ir_A de una tabla de análisis sintáctico LR para la gramática de expresiones (4.1), que repetimos a continuación con las producciones enumeradas:

$$\begin{array}{ll}
 (1) & E \rightarrow E + T \\
 (2) & E \rightarrow T \\
 (3) & T \rightarrow T * F \\
 (4) & T \rightarrow F \\
 (5) & T \rightarrow (E) \\
 (6) & F \rightarrow \text{id}
 \end{array}$$

Los códigos para las acciones son:

1. si significa desplazar y meter el estado i en la pila,
2. rj significa reducir mediante la producción enumerada como j ,
3. acc significa aceptar,
4. espacio en blanco significa error.

Observe que el valor de $\text{ir_A}[s, a]$ para el terminal a se encuentra en el campo ACCION conectado con la acción de desplazamiento en la entrada a , para el estado s . El campo ir_A proporciona $\text{ir_A}[s, A]$ para los no terminales A . Aunque no hemos explicado aún cómo se seleccionaron las entradas para la figura 4.37, en breve trataremos con esta cuestión.

ESTADO	ACCIÓN						ir_A		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Figura 4.37: Tabla de análisis sintáctico para la gramática de expresiones

En la entrada **id** * **id** + **id**, la secuencia del contenido de la pila y de la entrada se muestra en la figura 4.38. Para fines de claridad, también se muestran las secuencias de los símbolos gramaticales que corresponden a los estados contenidos en la pila. Por ejemplo, en la línea (1) el analizador LR se encuentra en el estado 0, el estado inicial sin símbolo gramatical, y con **id** el primer símbolo de entrada. La acción en la fila 0 y la columna **id** del campo acción de la figura 4.37 es s5, lo cual significa desplazar metiendo el estado 5. Esto es lo que ha ocurrido en la línea (2) se ha metido en la pila el símbolo de estado 5, mientras que **id** se ha eliminado de la entrada.

Después, * se convierte en el símbolo de entrada actual, y la acción del estado 5 sobre la entrada * es reducir mediante $F \rightarrow \mathbf{id}$. Se saca un símbolo de estado de la pila. Después se expone el estado 0. Como el ir_A del estado 0 en F es 3, el estado 3 se mete a la pila. Ahora tenemos la configuración de la línea (3). Cada uno de los movimientos restantes se determina en forma similar. $\square$

4.6.4 Construcción de tablas de análisis sintáctico SLR

El método SLR para construir tablas de análisis sintáctico es un buen punto inicial para estudiar el análisis sintáctico LR. Nos referiremos a la tabla de análisis sintáctico construida por este método como una tabla SLR, y a un analizador sintáctico LR que utiliza una tabla de análisis sintáctico SLR como un analizador sintáctico SLR. Los otros dos métodos aumentan el método SLR con información de anticipación.

El método SLR empieza con elementos LR(0) y un autómata LR(0), que presentamos en la sección 4.5. Es decir, dada una gramática G , la aumentamos para producir G' , con un nuevo símbolo inicial S' . A partir de G' construimos a C , la colección canónica de conjuntos de elementos para G' , junto con la función ir_A.

	PILA	SÍMBOLOS	ENTRADA	ACCIÓN
(1)	0		id * id + id \$	desplazar
(2)	0 5	id	* id + id \$	reducir mediante $F \rightarrow \mathbf{id}$
(3)	0 3	F	* id + id \$	reducir mediante $T \rightarrow F$
(4)	0 2	T	* id + id \$	desplazar
(5)	0 2 7	$T *$	id + id \$	desplazar
(6)	0 2 7 5	$T * \mathbf{id}$	+ id \$	reducir mediante $F \rightarrow \mathbf{id}$
(7)	0 2 7 10	$T * F$	+ id \$	reducir mediante $T \rightarrow T * F$
(8)	0 2	T	+ id \$	reducir mediante $E \rightarrow T$
(9)	0 1	E	+ id \$	desplazar
(10)	0 1 6	$E +$	id \$	desplazar
(11)	0 1 6 5	$E + \mathbf{id}$	\$	reducir mediante $F \rightarrow \mathbf{id}$
(12)	0 1 6 3	$E + F$	\$	reducir mediante $T \rightarrow F$
(13)	0 1 6 9	$E + T$	\$	reducir mediante $E \rightarrow E + T$
(14)	0 1	E	\$	aceptar

Figura 4.38: Movimientos de un analizador sintáctico LR con **id * id + id**

Después, las entradas ACCION e ir_A en la tabla de análisis sintáctico se construyen utilizando el siguiente algoritmo. Para ello, requerimos conocer SIGUIENTE(A) para cada no terminal A de una gramática (vea la sección 4.4).

Algoritmo 4.46: Construcción de una tabla de análisis sintáctico SLR.

ENTRADA: Una gramática aumentada G' .

SALIDA: Las funciones ACCION e ir_A para G' de la tabla de análisis sintáctico SLR.

MÉTODO:

1. Construir $C = \{ I_0, I_1, \dots, I_n \}$, la colección de conjuntos de elementos LR(0) para G' .
2. El estado i se construye a partir de I_i . Las acciones de análisis sintáctico para el estado i se determinan de la siguiente forma:
 - (a) Si $[A \rightarrow \alpha \cdot a \beta]$ está en I_i e $\text{ir_A}(I_i, a) = I_j$, entonces establecer $\text{ACCION}[i, a]$ a “desplazar j ”. Aquí, a debe ser una terminal.
 - (b) Si $[A \rightarrow \alpha \cdot]$ está en I_i , entonces establecer $\text{ACCION}[i, a]$ a “reducir $A \rightarrow \alpha$ ” para toda a en $\text{SIGUIENTE}(A)$; aquí, A tal vez no sea S' .
 - (c) Si $[S' \rightarrow S \cdot]$ está en I_i , entonces establecer $\text{ACCION}[i, \$]$ a “aceptar”.

Si resulta cualquier acción conflictiva debido a las reglas anteriores, decimos que la gramática no es SLR(1). El algoritmo no produce un analizador sintáctico en este caso.

3. Las transiciones de ir_A para el estado i se construyen para todos los no terminales A , usando la regla: Si $\text{ir_A}(I_i, A) = I_j$, entonces $\text{ir_A}[i, A] = j$.
4. Todas las entradas que no estén definidas por las reglas (2) y (3) se dejan como “error”.
5. El estado inicial del analizador sintáctico es el que se construyó a partir del conjunto de elementos que contienen $[S' \rightarrow \cdot S]$.

□

A la tabla de análisis sintáctico que consiste en las funciones ACCION e ir_A determinadas por el algoritmo 4.46 se le conoce como la *tabla SLR(1) para G* . A un analizador sintáctico LR que utiliza la tabla $\text{SLR}(1)$ para G se le conoce como analizador sintáctico $\text{SLR}(1)$ par G , y a una gramática que tiene una tabla de análisis sintáctico $\text{SLR}(1)$ se le conoce como *SLR(1)*. Por lo general, omitimos el “(1)” después de “SLR”, ya que no trataremos aquí con los analizadores sintácticos que tienen más de un símbolo de preanálisis.

Ejemplo 4.47: Vamos a construir la tabla SLR para la gramática de expresiones aumentada. La colección canónica de conjuntos de elementos $\text{LR}(0)$ para la gramática se mostró en la figura 4.31. Primero consideremos el conjunto de elementos I_0 :

$$\begin{aligned} E' &\rightarrow \cdot E \\ E &\rightarrow \cdot E + T \\ E &\rightarrow \cdot T \\ T &\rightarrow \cdot T * F \\ T &\rightarrow \cdot F \\ F &\rightarrow \cdot (E) \\ F &\rightarrow \cdot \text{id} \end{aligned}$$

El elemento $F \rightarrow \cdot (E)$ produce la entrada $\text{ACCION}[0, (] = \text{desplazar 4}$, y el elemento $F \rightarrow \cdot \text{id}$ produce la entrada $\text{ACCION}[0, \text{id}] = \text{desplazar 5}$. Los demás elementos en I_0 no producen ninguna acción. Ahora consideremos I_1 :

$$\begin{aligned} E' &\rightarrow E \cdot \\ E &\rightarrow E \cdot + T \end{aligned}$$

El primer elemento produce $\text{ACCION}[1, \$] = \text{aceptar}$, y el segundo produce $\text{ACCION}[1, +] = \text{desplazar 6}$. Ahora consideremos I_2 :

$$\begin{aligned} E &\rightarrow T \cdot \\ T &\rightarrow T \cdot * F \end{aligned}$$

Como $\text{SIGUIENTE}(E) = \{\$, +,)\}$, el primer elemento produce lo siguiente:

$$\text{ACCION}[2, \$] = \text{ACCION}[2, +] = \text{ACCION}[2,)] = \text{reducir } E \rightarrow T$$

El segundo elemento produce $\text{ACCION}[2, *] = \text{desplazar 7}$. Si continuamos de esta forma obtendremos las tablas ACCION y ir_A que se muestran en la figura 4.31. En esa figura, los números de las producciones en las acciones de reducción son los mismos que el orden en el que aparecen en la gramática original (4.1). Es decir, $E \rightarrow E + T$ es el número 1, $E \rightarrow T$ es 2, y así sucesivamente. □

Ejemplo 4.48: Cada una de las gramáticas SLR(1) no tiene ambigüedad, pero hay muchas gramáticas sin ambigüedad que no son SLR(1). Considere la gramática con las siguientes producciones:

$$\begin{aligned} S &\rightarrow L = R \mid R \\ L &\rightarrow *R \mid \mathbf{id} \\ R &\rightarrow L \end{aligned} \tag{4.49}$$

Consideremos que L y R representan *l-value* y *r-value*, respectivamente, y que $*$ es un operador que indica “el contenido de”.⁵ La colección canónica de conjuntos de elementos LR(0) para la gramática (4.49) se muestra en la figura 4.39.

$$\begin{array}{ll} I_0: & S' \rightarrow \cdot S \\ & S \rightarrow \cdot L = R \\ & S \rightarrow \cdot R \\ & L \rightarrow \cdot * R \\ & L \rightarrow \cdot \mathbf{id} \\ & R \rightarrow \cdot L \\ I_1: & S' \rightarrow S \cdot \\ I_2: & S \rightarrow L \cdot = R \\ & R \rightarrow L \cdot \\ I_3: & S \rightarrow R \cdot \\ I_4: & L \rightarrow * \cdot R \\ & R \rightarrow \cdot L \\ & L \rightarrow \cdot * R \\ & L \rightarrow \cdot \mathbf{id} \\ I_5: & L \rightarrow \mathbf{id} \cdot \\ I_6: & S \rightarrow L = \cdot R \\ & R \rightarrow \cdot L \\ & L \rightarrow \cdot * R \\ & L \rightarrow \cdot \mathbf{id} \\ I_7: & L \rightarrow * R \cdot \\ I_8: & R \rightarrow L \cdot \\ I_9: & S \rightarrow L = R \cdot \end{array}$$

Figura 4.39: Colección LR(0) canónica para la gramática (4.49)

Considere el conjunto de elementos I_2 . El primer elemento en este conjunto hace que ACCION[2, =] sea “desplazar 6”. Como SIGUIENTE(R) contiene = (para ver por qué, considere la derivación $S \Rightarrow L = R \Rightarrow *R = R$), el segundo elemento establece ACCION[2, =] a “reducir $R \rightarrow L$ ”. Como hay tanto una entrada de desplazamiento como una de reducción en ACCION[2, =], el estado 2 tiene un conflicto de desplazamiento/reducción con el símbolo de entrada =.

La gramática (4.49) no es ambigua. Este conflicto de desplazamiento/reducción surge del hecho de que el método de construcción del analizador sintáctico SLR no es lo bastante poderoso como para recordar el suficiente contexto a la izquierda para decidir qué acción debe realizar el analizador sintáctico sobre la entrada =, habiendo visto una cadena que puede reducirse a L . Los métodos canónico y LALR, que veremos a continuación, tendrán éxito en una colección más extensa de gramáticas, incluyendo la gramática (4.49). Sin embargo, observe que

⁵Al igual que en la sección 2.8.3, un *l-value* designa una ubicación y un *r-value* es un valor que puede almacenarse en una ubicación.

hay gramáticas sin ambigüedad para las cuales todos los métodos de construcción de analizadores sintácticos LR producirán una tabla de acciones de análisis sintáctico con conflictos. Por fortuna, dichas gramáticas pueden, por lo general, evitarse en las aplicaciones de los lenguajes de programación. $\square$

4.6.5 Prefijos viables

¿Por qué pueden usarse los autómatas LR(0) para realizar decisiones de desplazamiento-reducción? El autómata LR(0) para una gramática caracteriza las cadenas de símbolos gramaticales que pueden aparecer en la pila de un analizador sintáctico de desplazamiento-reducción para la gramática. El contenido de la pila debe ser un prefijo de una forma de frase derecha. Si la pila contiene a α y el resto de la entrada es x , entonces una secuencia de reducciones llevará a αx a S . En términos de derivaciones, $S \xRightarrow{rm}^* \alpha x$.

Sin embargo, no todos los prefijos de las formas de frases derechas pueden aparecer en la pila, ya que el analizador sintáctico no debe desplazar más allá del mango. Por ejemplo, suponga que:

$$E \xRightarrow{rm}^* F * \mathbf{id} \xRightarrow{rm}^* (E) * \mathbf{id}$$

Entonces, en diversos momentos durante el análisis sintáctico, la pila contendrá $($, $(E$ y (E) , pero no debe contener $(E)*$, ya que (E) es un mango, que el analizador sintáctico debe reducir a F antes de desplazar a $*$.

Los prefijos de las formas de frases derechas que pueden aparecer en la pila de un analizador sintáctico de desplazamiento-reducción se llaman *prefijos viables*. Se definen de la siguiente manera: un prefijo viable es un prefijo de una forma de frase derecha que no continúa más allá del extremo derecho del mango por la derecha de esa forma de frase. Mediante esta definición, siempre es posible agregar símbolos terminales al final de un prefijo viable para obtener una forma de frase derecha.

El análisis sintáctico SLR se basa en el hecho de que los autómatas LR(0) reconocen los prefijos viables. Decimos que el elemento $A \rightarrow \beta_1 \cdot \beta_2$ es *válido* para un prefijo viable $\alpha\beta_1$ si hay una derivación $S' \xRightarrow{rm}^* \alpha A w \Rightarrow \alpha\beta_1\beta_2 w$. En general, un elemento será válido para muchos prefijos viables.

El hecho de que $A \rightarrow \beta_1 \cdot \beta_2$ sea válido para $\alpha\beta_1$ nos dice mucho acerca de si debemos desplazar o reducir cuando encontremos a $\alpha\beta_1$ en la pila de análisis sintáctico. En especial, si $\beta_2 \neq \epsilon$, entonces sugiere que no hemos desplazado aún el mango hacia la pila, por lo que el desplazamiento es nuestro siguiente movimiento. Si $\beta_2 = \epsilon$, entonces parece que $A \rightarrow \beta_1$ es el mango, y debemos reducir mediante esta producción. Desde luego que dos elementos válidos pueden indicarnos que debemos hacer distintas cosas para el mismo prefijo viable. Podemos resolver algunos de estos conflictos analizando el siguiente símbolo de entrada, y otros podemos resolverlos mediante los métodos de la sección 4.8, pero no debemos suponer que todos los conflictos de acciones de análisis sintáctico pueden resolverse si se aplica el método LR a una gramática arbitraria.

Podemos calcular con facilidad el conjunto de elementos válidos para cada prefijo viable que puede aparecer en la pila de un analizador sintáctico LR. De hecho, un teorema central de la teoría de análisis sintáctico LR nos dice que el conjunto de elementos válidos para un prefijo viable γ es exactamente el conjunto de elementos a los que se llega desde el estado inicial,

Los elementos como estados de un AFN

Podemos construir un autómata finito no determinista N para reconocer prefijos viables si tratamos a los mismos elementos como estados. Hay una transición desde $A \rightarrow \alpha \cdot X \beta$ hacia $A \rightarrow \alpha X \cdot \beta$ etiquetada como X , y hay una transición desde $A \rightarrow \alpha \cdot B \beta$ hacia $B \rightarrow \cdot \gamma$ etiquetada como ϵ . Entonces, $\text{CERRADURA}(I)$ para el conjunto de elementos (estados de N) I es exactamente el cierre ϵ de un conjunto de estados de un AFN definidos en la sección 3.7.1. Por ende, $\text{ir_A}(I, X)$ proporciona la transición proveniente de I en el símbolo X del AFD construido a partir de N mediante la construcción del subconjunto. Si lo vemos de esta forma, el procedimiento $\text{elementos}(G')$ en la figura 4.33 es sólo la construcción del mismo subconjunto que se aplica al AFN N , con los elementos como estados.

a lo largo de la ruta etiquetada como γ en el autómata $\text{LR}(0)$ para la gramática. En esencia, el conjunto de elementos válidos abarca toda la información útil que puede deducirse de la pila. Aunque aquí no demostraremos este teorema, vamos a ver un ejemplo.

Ejemplo 4.50: Vamos a considerar de nuevo la gramática de expresiones aumentada, cuyos conjuntos de elementos y la función ir_A se exhiben en la figura 4.31. Sin duda, la cadena $E + T^*$ es un prefijo viable de la gramática. El autómata de la figura 4.31 se encontrará en el estado 7, después de haber leído $E + T^*$. El estado 7 contiene los siguientes elementos:

$$\begin{aligned} T &\rightarrow T * \cdot F \\ F &\rightarrow \cdot (E) \\ F &\rightarrow \cdot \text{id} \end{aligned}$$

que son precisamente los elementos válidos para $E + T^*$. Para ver por qué, considere las siguientes tres derivaciones por la derecha:

$$\begin{array}{lll} E' \Rightarrow E & E' \Rightarrow E & E' \Rightarrow E \\ \text{rm} \Rightarrow E + T & \text{rm} \Rightarrow E + T & \text{rm} \Rightarrow E + T \\ \text{rm} \Rightarrow E + T * F & \text{rm} \Rightarrow E + T * F & \text{rm} \Rightarrow E + T * F \\ \text{rm} \Rightarrow E + T * (E) & \text{rm} \Rightarrow E + T * (E) & \text{rm} \Rightarrow E + T * \text{id} \\ \text{rm} & \text{rm} & \text{rm} \end{array}$$

La primera derivación muestra la validez de $T \rightarrow T * \cdot F$, la segunda muestra la validez de $F \rightarrow \cdot (E)$, y la tercera muestra la validez de $F \rightarrow \cdot \text{id}$. Se puede mostrar que no hay otros elementos válidos para $E + T^*$, aunque aquí no demostraremos ese hecho. $\square$

4.6.6 Ejercicios para la sección 4.6

Ejercicio 4.6.1: Describa todos los prefijos viables para las siguientes gramáticas:

- a) La gramática $S \rightarrow 0 \mid S1 \mid 01$ del ejercicio 4.2.2(a).

! b) La gramática $S \rightarrow S S + \mid S S * \mid a$ del ejercicio 4.2.1.

! c) La gramática $S \rightarrow S (S) \mid \epsilon$ del ejercicio 4.2.2(c).

Ejercicio 4.6.2: Construya los conjuntos SLR de elementos para la gramática (aumentada) del ejercicio 4.2.1. Calcule la función ir_A para estos conjuntos de elementos. Muestre la tabla de análisis sintáctico para esta gramática. ¿Es una gramática SLR?

Ejercicio 4.6.3: Muestre las acciones de su tabla de análisis sintáctico del ejercicio 4.6.2 sobre la entrada $aa * a+$.

Ejercicio 4.6.4: Para cada una de las gramáticas (aumentadas) del ejercicio 4.2.2(a)-(g):

- a) Construya los conjuntos SLR de elementos y su función ir_A .
- b) Indique cualquier conflicto de acciones en sus conjuntos de elementos.
- c) Construya la tabla de análisis sintáctico SLR, si es que existe.

Ejercicio 4.6.5: Muestre que la siguiente gramática:

$$\begin{aligned} S &\rightarrow A a A b \mid B b B a \\ A &\rightarrow \epsilon \\ B &\rightarrow \epsilon \end{aligned}$$

es LL(1), pero no SLR(1).

Ejercicio 4.6.6: Muestre que la siguiente gramática:

$$\begin{aligned} S &\rightarrow S A \mid A \\ A &\rightarrow a \end{aligned}$$

es SLR(1), pero no LL(1).

!! Ejercicio 4.6.7: Considere la familia de gramáticas G_n definidas por:

$$\begin{aligned} S &\rightarrow A_i b_i && \text{para } 1 \leq i \leq n \\ A_i &\rightarrow a_j A_i \mid a_j && \text{para } 1 \leq i, j \leq n \text{ e } i \neq j \end{aligned}$$

Muestre que:

- a) G_n tiene $2n^2 - 2$ producciones.
- b) G_n tiene $2n^2 + n^2 + n$ conjuntos de elementos LR(0).
- c) G_n es SLR(1).

¿Qué dice este análisis acerca de la extensión que pueden llegar a tener los analizadores sintácticos LR?

! Ejercicio 4.6.8: Sugerimos que los elementos individuales pudieran considerarse como estados de un autómata finito no determinista, mientras que los conjuntos de elementos válidos son los estados de un autómata finito determinista (vea el recuadro titulado “Los elementos como estados de un AFN” en la sección 4.6.5). Para la gramática $S \rightarrow S S + \mid S S * \mid a$ del ejercicio 4.2.1:

- a) Dibuje el diagrama de transición (AFN) para los elementos válidos de esta gramática, de acuerdo a la regla que se proporciona en el recuadro antes citado.
- b) Aplique la construcción de subconjuntos (Algoritmo 3.20) a su AFN, a partir de (a). ¿Cómo se compara el AFD resultante con el conjunto de elementos LR(0) para la gramática?
- !!** c) Muestre que en todos los casos, la construcción de subconjuntos que se aplica al AFN que proviene de los elementos válidos para una gramática produce los conjuntos LR(0) de elementos.

! Ejercicio 4.6.9: La siguiente es una gramática ambigua:

$$\begin{aligned} S &\rightarrow A S \mid b \\ A &\rightarrow S A \mid a \end{aligned}$$

Construya para esta gramática su colección de conjuntos de elementos LR(0). Si tratamos de construir una tabla de análisis sintáctico LR para la gramática, hay ciertas acciones en conflicto. ¿Qué son? Suponga que tratamos de usar la tabla de análisis sintáctico eligiendo en forma no determinista una posible acción, cada vez que haya un conflicto. Muestre todas las posibles secuencias de acciones con la entrada *abab*.

4.7 Analizadores sintácticos LR más poderosos

En esta sección vamos a extender las técnicas anteriores de análisis sintáctico LR, para usar un símbolo de preanálisis en la entrada. Hay dos métodos distintos:

1. El método “LR canónico”, o simplemente “LR”, que utiliza al máximo el (los) símbolo(s) de preanálisis. Este método utiliza un extenso conjunto de elementos, conocidos como elementos LR(1).
2. El método “LR con símbolo de preanálisis” o “LALR(lookahead LR)”, que se basa en los conjuntos de elementos LR(0), y tiene mucho menos estados que los analizadores sintácticos comunes, basados en los elementos LR(1). Si introducimos con cuidado lecturas anticipadas en los elementos LR(0), podemos manejar muchas gramáticas más con el método LALR que con el SLR, y construir tablas de análisis sintáctico que no sean más grandes que las tablas SLR. LALR es el método de elección en la mayoría de las situaciones.

Después de presentar ambos métodos, concluiremos con una explicación acerca de cómo compactar las tablas de análisis sintáctico LR para los entornos con memoria limitada.